

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_49c1018a-e67\agent-a3a4bc3405adb215b.jsonl`
- SHA-256: `7606758931a1351496fa65fadd2361778b0476a1059bcf4d45e2294810db4788`
- Source modified: `2026-06-18T01:36:14+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.223Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: numeric / statistical / off-by-one.

Across eval + pipeline windowing: div-by-zero, n=0/1 means, NaN/inf, fps/frame<->time rounding, interval boundary inclusivity, IoU/tolerance match edges, bootstrap/sign-test/CI math, cap/trim boundary conditions.

2. assistant (2026-06-18T01:33:14.555Z)

I'll start by exploring the eval and pipeline windowing code to hunt for numeric/statistical/off-by-one bugs. Let me pull the repo first and map the relevant files.

3. user (2026-06-18T01:33:16.729Z)

4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)

4. user (2026-06-18T01:33:16.961Z)

.claude\worktrees\hunt-base\backend\eval\ablation.py
.claude\worktrees\hunt-base\backend\eval__init__.py
.claude\worktrees\hunt-base\backend\eval\annotations.py
.claude\worktrees\hunt-base\backend\eval\audio\e1_probe.py
.claude\worktrees\hunt-base\backend\eval\audio__init__.py
.claude\worktrees\hunt-base\backend\eval\batch.py
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py
.claude\worktrees\hunt-base\backend\eval\calibrate_wasb.py
.claude\worktrees\hunt-base\backend\eval\calibration.py
.claude\worktrees\hunt-base\backend\eval\classifier.py
.claude\worktrees\hunt-base\backend\eval\distill_local.py
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py
.claude\worktrees\hunt-base\backend\eval\eval_stats.py
.claude\worktrees\hunt-base\backend\eval\experiment.py
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py

```
.claude\worktrees\hunt-base\backend\eval\gemini_refine.py
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py
.claude\worktrees\hunt-base\backend\eval\gt_loader.py
.claude\worktrees\hunt-base\backend\eval\harness.py
.claude\worktrees\hunt-base\backend\eval\metrics.py
.claude\worktrees\hunt-base\backend\eval\per_user_eval.py
.claude\worktrees\hunt-base\backend\eval\per_user_gate.py
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py
.claude\worktrees\hunt-base\backend\eval\promotion.py
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py
.claude\worktrees\hunt-base\backend\eval\regression.py
.claude\worktrees\hunt-base\backend\eval\score_calibration.py
.claude\worktrees\hunt-base\backend\eval\segmentation_metrics.py
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py
.claude\worktrees\hunt-base\backend\eval\splits.py
.claude\worktrees\hunt-base\backend\eval\tolerance_metrics.py
.claude\worktrees\hunt-base\backend\eval\training.py
.claude\worktrees\hunt-base\backend\eval>windowing.py
```

5. assistant (2026-06-18T01:33:22.653Z)

Let me focus on the core numeric/statistical files. I'll read the key metrics, tolerance, windowing, and stats files in parallel.

6. user (2026-06-18T01:33:23.468Z)

```
1      """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3      The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4      golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5      over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6      boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7      number (a single ? misleads; see the design doc).
8
9      Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10     asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11     rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13     Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14     else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15     eligibility graphs that rally intervals produce, so scores are environment-independent.
16     """
17     from __future__ import annotations
18
19     import statistics
20     from typing import Any, Dict, List, Optional, Tuple
21
22     Interval = Tuple[float, float]
23     #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24     Match = Tuple[int, int, float, float]
25
26     #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
```

```

27 #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
28 TAU_START: float = 2.0
29 TAU_END: float = 1.5
30 SWEEP_TAU: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
31
32
33 def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
34     return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
35
36
37 def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
38                       tau_start: float, tau_end: float) -> List[Match]:
39     """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
40 assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
41 graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
42     pairs: List[Tuple[float, float, int, int]] = []
43     for i, p in enumerate(preds):
44         for j, g in enumerate(gts):
45             if _eligible(p, g, tau_start, tau_end):
46                 # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
47                 pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
48     pairs.sort()
49     used_p: set = set()
50     used_g: set = set()
51     matches: List[Match] = []
52     for _, _, i, j in pairs:
53         if i in used_p or j in used_g:
54             continue
55         used_p.add(i)
56         used_g.add(j)
57         matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
58     matches.sort(key=lambda mm: mm[1])
59     return matches
60
61
62 def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
63                           tau_start: float, tau_end: float) -> Optional[List[Match]]:
64     """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
65 numpy/scipy are unavailable so the caller falls back to greedy."""
66     try:
67         import numpy as np
68         from scipy.optimize import linear_sum_assignment
69     except Exception:
70         return None
71     n, m = len(preds), len(gts)
72     size = max(n, m)
73     big = 1.0e6
74     cost = np.full((size, size), big, dtype=float)
75     for i, p in enumerate(preds):
76         for j, g in enumerate(gts):
77             if _eligible(p, g, tau_start, tau_end):
78                 cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
79     rows, cols = linear_sum_assignment(cost)
80     matches: List[Match] = []
81     for i, j in zip(rows.tolist(), cols.tolist()):
82         if i < n and j < m and cost[i, j] < big: # drop forced ineligible assignments
83             matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
84     matches.sort(key=lambda mm: mm[1])
85     return matches
86

```

```

87
88     def match_1to1(preds: List[Interval], gts: List[Interval],
89                   tau_start: float = TAU_START, tau_end: float = TAU_END
90                   ) -> Tuple[List[Match], float, float, float]:
91         """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
92         (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
93         present; deterministic greedy otherwise."""
94         n, m = len(preds), len(gts)
95         if n == 0 or m == 0:
96             return [], 0.0, 0.0, 0.0
97         matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
98         if matches is None:
99             matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
100         k = len(matches)
101         precision = k / n
102         recall = k / m
103         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
104         return matches, precision, recall, f1
105
106
107     def _percentile(xs: List[float], p: float) -> float:
108         """Linear-interpolated percentile (pure stdlib)."""
109         if not xs:
110             return 0.0
111         s = sorted(xs)
112         k = (len(s) - 1) * p / 100.0
113         lo = int(k)
114         hi = min(lo + 1, len(s) - 1)
115         return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118     def _overlap(a: Interval, b: Interval) -> float:
119         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122     def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124         end open" diagnostic that aims R4.
125
126         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128         ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131         def dist(vals: List[float]) -> Dict[str, float]:
132             av = [abs(v) for v in vals]
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []

```

```

142         for g in gts:
143             best = max(preds, key=lambda p: _overlap(p, g), default=None)
144             if best is not None and _overlap(best, g) > 0.0:
145                 ds.append(best[0] - g[0])
146                 de.append(best[1] - g[1])
147         return {"dstart": dist(ds), "dend": dist(de)}
148
149
150     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151         """The over-segmentation GUARD (per-video no-regression in promotion):
152         ``split_count`` (?2
153         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
154         bipartite
155         overlap-graph in ``segmentation_metrics.merge_split_report``.
156         from backend.eval.segmentation_metrics import merge_split_report
157         r = merge_split_report(preds, gts)
158         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
159
160     # ----- #
161     # R1 ? net over-segmentation promotion guard
162     # ----- #
163     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
164     one
165     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
166     granularity); a
167     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
168     strictly
169     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
170     R1 evidence;
171     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
172     WEIGHT_MERGE: float = 2.0
173     WEIGHT_SPLIT: float = 1.0
174
175     def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
176                       weight_merge: float = WEIGHT_MERGE, weight_split: float =
177     WEIGHT_SPLIT,
178                       use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
179         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
180         video rule).
181
182         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
183         already
184         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
185         (default) ?
186         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
187         (else by
188         position). Returns the promote-or-block verdict for promoting ``cand`` over
189         ``base``.
190
191         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
192         ``cand`` raises
193         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
194         cue (same
195         windowing structure, an increase = a real regression) but mis-fires on a
196         *structural* change
197         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
198         ``split_count`` from
199         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
200         rallies is
201         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
202         into pieces
203         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
204         rallies hidden).

```

```

188         So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
189         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
190         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH $6.)
191
192         **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193         require BOTH:
194         1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195         2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196         of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197         "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198         With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201         **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202         strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203         """
204         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205             if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206                 cmap = {c["name"]: c for c in cand}
207                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211         def _cost(row: Dict[str, Any]) -> float:
212             if use_rates:
213                 return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216         pairs = _aligned()
217         n = len(pairs)
218         base_costs = [_cost(b) for b, _ in pairs]
219         cand_costs = [_cost(c) for _, c in pairs]
220         base_net = (sum(base_costs) / n) if n else 0.0
221         cand_net = (sum(cand_costs) / n) if n else 0.0
222         net_delta = cand_net - base_net
223
224         # Per-video regressions, reported for transparency.
225         new_merges: List[Dict[str, Any]] = [] # raw COUNT increases (what strict
trips on)
226         new_splits: List[Dict[str, Any]] = [] # raw COUNT increases (what strict
trips on)
227         merge_rate_regress: List[Dict[str, Any]] = [] # RATE increases (the real over-
merge regression)
228         for b, c in pairs:
229             if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230                 new_merges.append({"name": b.get("name", ""),
231                                     "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232             if float(c["split_count"]) > float(b["split_count"]) + eps:
233                 new_splits.append({"name": b.get("name", ""),
234                                     "base": float(b["split_count"]), "cand":

```

```

float(c["split_count"])))
235         if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236             merge_rate_regress.append({"name": b.get("name", ""),
237                                         "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])}))
238
239     strict_passes = (not new_merges) and (not new_splits)
240     net_cost_ok = net_delta <= eps
241     no_merge_rate_regress = not merge_rate_regress
242     passes = bool(net_cost_ok and no_merge_rate_regress)
243     return {
244         "passes": passes,
245         "strict_passes": strict_passes,
246         "net_cost_ok": net_cost_ok,
247         "no_merge_rate_regress": no_merge_rate_regress,
248         "base_net": base_net,
249         "cand_net": cand_net,
250         "net_delta": net_delta,
251         "n": n,
252         "use_rates": use_rates,
253         "weight_merge": weight_merge,
254         "weight_split": weight_split,
255         "new_merges": new_merges,
256         "new_splits": new_splits,
257         "merge_rate_regress": merge_rate_regress,
258     }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                         tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                        taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278

```

7. user (2026-06-18T01:33:24.425Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13

```

```

14
15 def interval_iou(a: Interval, b: Interval) -> float:
16     """Temporal intersection-over-union of two intervals.
17
18     Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19     overlap / union, both measured as durations on the timeline.
20     """
21     a_start, a_end = a
22     b_start, b_end = b
23     if a_end <= a_start or b_end <= b_start:
24         return 0.0
25     inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26     if inter <= 0.0:
27         return 0.0
28     union = (a_end - a_start) + (b_end - b_start) - inter
29     return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34     """One matched predicted?ground-truth pair."""
35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
(misses)
46
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                     iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65             # Highest IoU first; ties broken by indices for determinism.
66             candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76

```



```

77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104 def score(preds: List[Interval], gts: List[Interval],
105          iou_threshold: float = 0.5,
106          match_result: Optional[MatchResult] = None) -> Score:
107     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110     extracting the failure lists); otherwise matching is done here.
111     """
112     mr = match_result if match_result is not None else match_intervals(preds, gts,
113                                iou_threshold)
114
115     tp = len(mr.matches)
116     fp = len(mr.unmatched_pred_indices)
117     fn = len(mr.unmatched_gt_indices)
118
119     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122     else 0.0
123
124     if mr.matches:
125         mean_iou = sum(m.iou for m in mr.matches) / tp
126         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127                               mr.matches) / tp
128         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129                               mr.matches) / tp
130     else:
131         mean_iou = mean_start_err = mean_end_err = 0.0
132
133     return Score(
134         iou_threshold=iou_threshold,
135         num_pred=len(preds), num_gt=len(gts),
136         true_positives=tp, false_positives=fp, false_negatives=fn,
137         precision=precision, recall=recall, f1=f1,
138         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139     )

```

```

138 def pr_curve(preds: List[Interval], gts: List[Interval],
139             thresholds: Optional[List[float]] = None) -> List[Score]:
140     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142     Lets you see strict-vs-lenient behaviour rather than a single operating point.
143     """
144     if thresholds is None:
145         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146     return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

8. user (2026-06-18T01:33:24.872Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4     ?
5     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6     built
7     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9     pipeline
10    (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11    merge_gap=2.0,
12    min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13    is
14    measured on candidates **production never serves**, so a "win" need not transfer. This
15    is
16    exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17    set
18    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
21    the
22    SERVED preset is **single-sourced from the same Pydantic config defaults production
23    reads**
24    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25    Helpers
26    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27    *actual* served windowing out of a live config (overrides included), so an A/B can
28    target the
29    operating point a given deployment truly serves.
30
31    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32    windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33    `per_user_gate`). This module is the **what windowing** ; that one is the **promotion
34    gate**.
35
36    Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
37    a
38    caller opts in (`distill_local` re-points its module-level presets here, byte-
39    identically).
40
41    """
42    from __future__ import annotations
43
44    from dataclasses import dataclass
45    from typing import Any, Dict, List, Mapping, Tuple
46
47    from backend.config.models import IndexingConfig
48
49    Interval = Tuple[float, float]
50
51    __all__ = [
52        "WindowingPreset",
53        "SERVED",

```

```

39     "PROPOSAL",
40     "PRESETS",
41     "served_windowing_from_config",
42     "build_windows",
43     "windows_to_intervals",
44 ]
45
46
47 @dataclass(frozen=True)
48 class WindowingPreset:
49     """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51     ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52     "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53     ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54     knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55     """
56
57     name: str
58     velocity_thresh: float
59     merge_gap: float
60     min_window_duration: float
61     #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62     #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63     max_window_duration: float = 0.0
64     #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65     #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66     hard_cap: bool = False
67     #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68     #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69     inactivity_end: float = 0.0
70     #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71     inactivity_rally_thresh: float = 0.08
72     inactivity_min_density: float = 0.34
73     #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74     #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75     #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76     #: factor keeps it surgical (a normal long rally is left alone).
77     blob_fallback: bool = False
78     blob_factor: float = 4.0
79
80     def as_tuple(self) -> Tuple[float, float, float]:
81         """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82         ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85

```

```

86     def to_dict(self) -> Dict[str, Any]:
87         return {
88             "name": self.name,
89             "velocity_thresh": self.velocity_thresh,
90             "merge_gap": self.merge_gap,
91             "min_window_duration": self.min_window_duration,
92             "max_window_duration": self.max_window_duration,
93             "hard_cap": self.hard_cap,
94             "inactivity_end": self.inactivity_end,
95             "inactivity_rally_thresh": self.inactivity_rally_thresh,
96             "inactivity_min_density": self.inactivity_min_density,
97             "blob_fallback": self.blob_fallback,
98             "blob_factor": self.blob_factor,
99         }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
104         reads.
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
107         wasb/fusion/tracknet)
108         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
109         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
110
111         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
112         the whole
113         point: change a served default in ``config/models.py`` and the eval SERVED preset
114         follows,
115         so the two can never silently diverge again.
116         """
117         idx = IndexingConfig() # construct with defaults ? the served operating point
118         return WindowingPreset(
119             name="served",
120             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
121             default 0.02
122             merge_gap=float(idx.dead_time_merge_gap),
123             min_window_duration=float(idx.min_action_window_duration),
124             max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
125             (default 0=OFF)
126             hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
127             (default OFF)
128             inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
129             (default 0=OFF)
130             inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
131             inactivity_min_density=float(idx.inactivity_min_density),
132             blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
133             (default OFF)
134             blob_factor=float(idx.window_blob_factor),
135             )
136
137     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
138     cue is
139
140     #: only promotable if it holds up HERE ? the operating point real users get.
141     SERVED: WindowingPreset = _served_preset_from_defaults()
142
143     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
144     (deliberately
145     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
146     ? a
147
148     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
149     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
150     valid.

```

```

138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
150         an arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
151         ``family`` (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
152         served default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
153         operating point; this is *this config's* operating point. Falls back to the served defaults for any
154         absent key.
155         """
156         idx = _as_mapping(config, "indexing")
157         family_cfg = _as_mapping(idx, family)
158         return WindowingPreset(
159             name=f"served:{family}",
160             velocity_thresh=float(family_cfg.get("velocity_threshold",
161 SERVED.velocity_thresh)),
162             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
163             min_window_duration=float(idx.get("min_action_window_duration",
164 SERVED.min_window_duration)),
165             max_window_duration=float(idx.get("max_window_duration",
166 SERVED.max_window_duration)),
167             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
168             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
169             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
170 SERVED.inactivity_rally_thresh)),
171             inactivity_min_density=float(idx.get("inactivity_min_density",
172 SERVED.inactivity_min_density)),
173             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
174             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
175         )
176
177     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
178         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
179         section
180         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
181         mapping."""
182         if obj is None:
183             return {}
184         if isinstance(obj, Mapping):
185             sub = obj.get(key, {})
186         elif hasattr(obj, "get"):
187             sub = obj.get(key, {})
188         else:
189             sub = getattr(obj, key, {})
190         return sub if isinstance(sub, Mapping) else {}
191
192     def build_windows(points: List[Any], fps: float, frame_width: float,
193         preset: WindowingPreset) -> List[Dict[str, Any]]:
194         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
195         call site).
196     """

```

```

190     A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191     instead of three loose floats, so eval candidate-building and the served path
reference the
192     SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195     vt, mg, mwd = preset.as_tuple()
196     return TrackNetRunner.trajectory_to_action_windows(
197         points, fps=fps, frame_width=frame_width,
198         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199         max_window_duration=preset.max_window_duration,
200         window_hard_cap=preset.hard_cap,
201         window_inactivity_end=preset.inactivity_end,
202         inactivity_rally_thresh=preset.inactivity_rally_thresh,
203         inactivity_min_density=preset.inactivity_min_density,
204         window_blob_fallback=preset.blob_fallback,
205         window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

9. user (2026-06-18T01:33:25.755Z)

```

1     """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3     The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4     single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5     and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6     ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7     turns those into an honest summary:
8
9     - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10     interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11     many *videos* it wins on (the distribution-free test the repo already leans on:
12     "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14     producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15     match, never a held-out window from the same video).
16
17     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20     """
21     from __future__ import annotations
22
23     import math
24     import random
25     import statistics
26     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28     Interval = Tuple[float, float]
29     FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31

```

```

32 def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                 n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34     """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36     ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37     resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38     """
39     vals = [float(v) for v in values]
40     n = len(vals)
41     if n == 0:
42         return (0.0, 0.0)
43     if n == 1:
44         return (vals[0], vals[0])
45     rng = random.Random(seed)
46     means: List[float] = []
47     for _ in range(max(1, n_boot)):
48         means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49     means.sort()
50     alpha = (1.0 - confidence) / 2.0
51     last = len(means) - 1
52     lo = means[max(0, int(math.floor(alpha * last)))]
53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57 def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59     """{mean, std, n, ci_low, ci_high, confidence} ? report this, never a bare mean
(C1)."""
60     vals = [float(v) for v in values]
61     n = len(vals)
62     mean = statistics.mean(vals) if vals else 0.0
63     std = statistics.pstdev(vals) if n > 1 else 0.0
64     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65     return {"mean": mean, "std": std, "n": float(n),
66           "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*?" check.
75     """
76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87           "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

```

```

94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
96     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
97     test agrees ? not when a single mean ticked up.
98     """
99     n = min(len(a_values), len(b_values))
100     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
101     summary = mean_with_ci(deltas, confidence, n_boot, seed)
102     return {
103         "mean_delta": summary["mean"],
104         "ci_low": summary["ci_low"],
105         "ci_high": summary["ci_high"],
106         "n": float(n),
107         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108         "sign_test": paired_sign_test(deltas, eps),
109     }
110
111
112 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116     first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117
118     """
119     seen: List[Any] = []
120     for g in groups:
121         if g not in seen:
122             seen.append(g)
123     n = len(groups)
124     folds: List[Tuple[List[int], List[int]]] = []
125     for g in seen:
126         test = [i for i in range(n) if groups[i] == g]
127         train = [i for i in range(n) if groups[i] != g]
128         if test and train:
129             folds.append((train, test))
130     return folds
131
132 def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133     """Out-of-fold predictions for honest stacking (C3).
134
135     For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136     other groups and returns predictions for ``test_idx`` (in that order); results are
137     scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138     when used as arbiter features, never include in-sample predictions ? the leakage
path
139     that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140     (needs >= 2 groups to predict).
141     """
142     n = len(groups)
143     oof: List[Optional[Any]] = [None] * n
144     for train_idx, test_idx in grouped_folds(groups):
145         preds = fit_predict(train_idx, test_idx)
146         for pos, ti in enumerate(test_idx):
147             if pos < len(preds):
148                 oof[ti] = preds[pos]
149     return oof

```


10. assistant (2026-06-18T01:33:28.436Z)

Let me look at the windowing implementation in the detector runner, and the segmentation_metrics and other statistical files.

11. user (2026-06-18T01:33:29.398Z)

```

1      """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3      Temporal IoU (`metrics.score`) is nearly blind to *fragmentation*: a single rally
4      chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5      still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6      literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7      thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8      sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9      tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10     structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12     This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13     changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14     ?
15     so it runs in the GPU-free / numpy-free lanes too.
16
17     Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
18     A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
19     scorer
20     emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
21
22     Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
23     the
24     ordered sequence of segment *labels* and is degenerate for a single action class ?
25     rally/
26     background segments strictly alternate, so the edit distance collapses to (twice) the
27     segment-count difference, adding nothing over ``segment_count_ratio``. So we
28     intentionally
29     omit it and instead make over-segmentation explicit with ``merge_split_report`` (a
30     bipartite
31     overlap-graph breakdown into merge/split/matched/missed/spurious ? see below), alongside
32     ``f1_at_overlaps`` (F1@k) + ``mean_average_precision`` (mAP@tIoU). Those, not a single-
33     class
34     edit score, are the over-segmentation-sensitive metrics for this binary rally setting.
35     """
36     from __future__ import annotations
37
38     from typing import Dict, List, Sequence, Tuple
39
40     from backend.eval.metrics import Interval, interval_iou, score
41
42     #: A predicted interval plus the confidence used to rank it for average precision.
43     ScoredInterval = Tuple[Interval, float]
44
45     #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
46     DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
47
48     #: Default tIoU sweep for mean average precision ? the action-detection convention.
49     DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
50
51     def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
52                       overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
53         """Segmental F1 at each overlap threshold ({overlap: f1}) ? the TAS F1@k.
54
55         Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
56         for 1 rally) scores extra false positives and a merged prediction misses a rally ?

```

```

50     both drag F1 down where temporal-IoU alone would not flinch.
51     """
52     return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
53
54
55 def average_precision(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
56                       iou_threshold: float = 0.5) -> float:
57     """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
58
59     Predictions are ranked by descending confidence; each is a true positive iff it
60     claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
61     available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
62     there are no ground-truth rallies or no predictions reach the threshold.
63     """
64     n_gt = len(gts)
65     if n_gt == 0 or not scored_preds:
66         return 0.0
67
68     order = sorted(range(len(scored_preds)),
69                   key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by
index
70     matched_gt = set()
71     precisions: List[float] = []
72     recalls: List[float] = []
73     cum_tp = 0
74     cum_fp = 0
75     for i in order:
76         interval, _conf = scored_preds[i]
77         best_iou, best_gi = 0.0, -1
78         for gi, g in enumerate(gts):
79             if gi in matched_gt:
80                 continue
81             iou = interval_iou(interval, g)
82             if iou >= iou_threshold and iou > best_iou:
83                 best_iou, best_gi = iou, gi
84         if best_gi >= 0:
85             matched_gt.add(best_gi)
86             cum_tp += 1
87         else:
88             cum_fp += 1
89     precisions.append(cum_tp / (cum_tp + cum_fp))
90     recalls.append(cum_tp / n_gt)
91
92     # Monotone precision envelope (take the max precision to the right), then integrate
93     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
94     for i in range(len(precisions) - 2, -1, -1):
95         precisions[i] = max(precisions[i], precisions[i + 1])
96     ap = 0.0
97     prev_recall = 0.0
98     for p, r in zip(precisions, recalls):
99         if r > prev_recall:
100             ap += p * (r - prev_recall)
101             prev_recall = r
102     return ap
103
104
105 def mean_average_precision(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
106                           iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
107     """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
108     tious = list(iou_thresholds)
109     if not tious:
110         return 0.0
111     return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
112
113

```

```

114 def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
115     """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
116
117     ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
118     merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
119     ``gts``
120     is empty (use F1@k / mAP for the graded picture).
121     """
122     if not gts:
123         return 0.0
124     return len(preds) / len(gts)
125
126 def _overlaps(a: Interval, b: Interval) -> bool:
127     """True iff two intervals share any positive temporal extent."""
128     return min(a[1], b[1]) - max(a[0], b[0]) > 0.0
129
130
131 def merge_split_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, float]:
132     """Explicit over-/under-segmentation breakdown via the bipartite overlap graph.
133
134     Where ``segment_count_ratio`` only compares *counts* and ``f1_at_overlaps`` blends
135     merge
136     and split into one F1, this names the two failure modes directly ? the diagnostic
137     that
138     surfaced our local-segmenter over-MERGE (one giant window swallowing many rallies).
139     Build
140     the bipartite graph (edge = any positive temporal overlap between a pred and a gt),
141     take its
142     connected components, and classify each by ``(#preds, #gts)``:
143
144     - ``(1,1)`` matched    ``(1,0)`` spurious (pred, no gt)    ``(0,1)`` missed (gt, no
145     pred)
146     - ``(1,>1)`` **merge** ? one predicted window covers ?2 rallies (over-MERGE; our
147     failure)
148     - ``(>1,1)`` **split** ? one rally cut across ?2 predictions (over-segmentation)
149     - ``(>1,>1)`` complex (tangled many-to-many)
150
151     Returns counts plus ``merge_rate`` (fraction of GT rallies swallowed inside a merge
152     group)
153     and ``split_rate`` (fraction of GT rallies cut by ?2 predictions) ? the two headline
154     over-segmentation rates. Pure, deterministic, stdlib-only.
155     """
156     n, m = len(preds), len(gts)
157     p_adj: List[List[int]] = [[] for _ in range(n)]
158     g_adj: List[List[int]] = [[] for _ in range(m)]
159     for i, p in enumerate(preds):
160         for j, g in enumerate(gts):
161             if _overlaps(p, g):
162                 p_adj[i].append(j)
163                 g_adj[j].append(i)
164
165     seen_p: set = set()
166     seen_g: set = set()
167     comps: List[Tuple[set, set]] = []
168     for start in range(n):
169         if start in seen_p:
170             continue
171         ps: set = set()
172         gs: set = set()
173         stack: List[Tuple[str, int]] = [("p", start)]
174         while stack:
175             side, k = stack.pop()
176             if side == "p":
177                 if k in ps:

```

```

171         continue
172         ps.add(k)
173         seen_p.add(k)
174         stack.extend(("g", j) for j in p_adj[k] if j not in gs)
175     else:
176         if k in gs:
177             continue
178             gs.add(k)
179             seen_g.add(k)
180             stack.extend(("p", i) for i in g_adj[k] if i not in ps)
181         comps.append((ps, gs))
182     comps.extend((set(), {j}) for j in range(m) if j not in seen_g) # gts with no
overlapping pred
183
184     counts = {"matched": 0, "merges": 0, "splits": 0, "spurious": 0, "missed": 0,
"complex": 0}
185     gts_merged = gts_split = 0
186     for ps, gs in comps:
187         a, b = len(ps), len(gs)
188         if (a, b) == (1, 1):
189             counts["matched"] += 1
190         elif (a, b) == (1, 0):
191             counts["spurious"] += 1
192         elif (a, b) == (0, 1):
193             counts["missed"] += 1
194         elif a == 1 and b > 1:
195             counts["merges"] += 1
196             gts_merged += b
197         elif b == 1 and a > 1:
198             counts["splits"] += 1
199             gts_split += 1
200         else:
201             counts["complex"] += 1
202     out: Dict[str, float] = {k: float(v) for k, v in counts.items()}
203     out["merge_rate"] = (gts_merged / m) if m else 0.0
204     out["split_rate"] = (gts_split / m) if m else 0.0
205     return out
206
207
208     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
209                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
210                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
object]:
211         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio + merge/split
breakdown.
212
213         ``scored_preds`` carry confidences for AP; F1@k + the merge/split breakdown use the
bare
214         intervals. Returns a plain dict (JSON-friendly, string keys) so it can ride next to
a
215         ``compare`` result or land in the experiment leaderboard.
216         """
217         preds = [iv for iv, _conf in scored_preds]
218         return {
219             "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()},
220             "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
221             "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
222             "segment_count_ratio": segment_count_ratio(preds, gts),
223             "merge_split": merge_split_report(preds, gts),
224             "num_pred": len(preds),
225             "num_gt": len(gts),
226         }
227

```

12. user (2026-06-18T01:33:30.623Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True          # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"          # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800          # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False          # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0          # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47          tn = (idx_cfg or {}).get("tracknet", {}) or {}
48          return cls(
49              distro=tn.get("wsl_distro", "Ubuntu"),
50              repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51              conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52              conda_env=tn.get("conda_env", "TrackNetV4"),
53              weights_path=tn.get("weights_path", ""),
54              queue_length=int(tn.get("queue_length", 5)),
55              stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
```

```

56         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57         timeout_sec=int(tn.get("timeout_sec", 1800)),
58         keep_frames=bool(tn.get("keep_frames", False)),
59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU'))))\"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
112             except subprocess.TimeoutExpired:
113                 return False, "WSL healthcheck timed out."
114             out = (res.stdout or "").strip()
115             if res.returncode != 0:
116                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117             return True, out
118
119 # ----- inference

```

```

119
120 def run_predict(self, video_win_path: str, output_win_dir: str,
121                 video_id: Optional[str] = None) -> Optional[str]:
122     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
123
124     ``output_win_dir`` is a Windows directory (under the repo's output/) that
125     WSL writes into via its /mnt view, so the Windows process can read the CSV
126     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
127     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128     a filename cannot collide on the output CSV.
129     """
130     if not self.cfg.weights_path:
131         logger.error(
132             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133             "in config.json to the WSL path of your trained TrackNetV4 weights."
134         )
135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}__{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "

```

```

182         f"python predict.py "
183         f"--video_path {wsl_tilde_quote(wsl_video)} "
184         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185         f"--output_dir {wsl_tilde_quote(wsl_out)} "
186         f"--queue_length {self.cfg.queue_length}"
187     )
188     logger.info("Running TrackNet inference on %s ...", base)
189     try:
190         res = self._wsl_bash(cmd)
191     except subprocess.TimeoutExpired:
192         logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193         return None
194
195     if res.returncode != 0:
196         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197         return None
198
199     if not os.path.exists(expected_csv_win):
200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205     # regenerable intermediate). On earlier failure we return above without
cleaning.
206     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207         logger.info("Cache hygiene: removing staged video for %s "
208                     "(set indexing.tracknet.keep_frames=true to retain).", key)
209         self._wsl_rm_rf([wsl_video])
210     return expected_csv_win
211
212 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215     Returns the WSL path to the staged copy, or None on failure.
216     """
217     src_mnt = to_wsl_mnt_path(video_win_path)
218     reason = self.wsl_source_unreadable_reason(src_mnt)
219     if reason:
220         logger.error("Cannot stage video into WSL: %s", reason)
221         return None
222     dest = f"{self.cfg.wsl_stage_dir}/{base}"
223     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224     try:
225         res = self._wsl_bash(cmd)
226     except subprocess.TimeoutExpired:
227         logger.error("Staging video into WSL timed out.")
228         return None
229     if res.returncode != 0 or "OK" not in (res.stdout or ""):
230         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231         return None
232     return dest
233
234 # ----- CSV -> windows
235
236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:

```



```

241         reader = csv.DictReader(f)
242         for row in reader:
243             try:
244                 points.append(TrajectoryPoint(
245                     frame=int(row["Frame"]),
246                     visible=int(row["Visibility"]) == 1,
247                     x=float(row["X"]),
248                     y=float(row["Y"]),
249                 ))
250             except (KeyError, ValueError):
251                 continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively

```

```

296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
297         (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
300         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
301         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
302         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` * x
303         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
305         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
306         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309         """
310         if fps <= 0:
311             fps = 30.0
312         if frame_width <= 0:
313             frame_width = 1280.0
314
315         # Compute per-frame normalised velocity from the last *visible* point.
316         active = [] # (frame_idx, velocity)
317         prev: Optional[TrajectoryPoint] = None
318         for p in points:
319             if not p.visible:
320                 prev = None # break the trajectory; absent shuttle = not in flight
321                 continue
322             if prev is not None:
323                 dframes = p.frame - prev.frame
324                 if dframes > 0:
325                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                     velocity = (dist / frame_width) * (fps / dframes)
327                     if velocity > velocity_thresh:
328                         active.append((p.frame, velocity))
329             prev = p
330
331         if not active:
332             return []
333
334         max_gap_frames = int(merge_gap * fps)
335
336         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337             vels = [v for _, v in group]
338             return {
339                 "start": group[0][0] / fps + chunk_start,
340                 "end": group[-1][0] / fps + chunk_start,
341                 "active_frame_count": len(group),
342                 "peak_velocity": max(vels),
343                 "mean_velocity": sum(vels) / len(vels),
344                 "track_id_count": 1, # single shuttle; kept for window-shape parity
345                 "chunk_index": chunk_index,
346             }
347
348         groups: List[List[Tuple[int, float]]] = []
349         group = [active[0]]

```

```

350         for af in active[1:]:
351             if af[0] - group[-1][0] <= max_gap_frames:
352                 group.append(af)
353             else:
354                 groups.append(group)
355                 group = [af]
356         groups.append(group)
357
358         if max_window_duration > 0:
359             groups = TrackNetRunner._split_overlong_groups(
360                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                 hard_cap=window_hard_cap)
362
363         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368         if window_inactivity_end > 0:
369             tw = int(window_inactivity_end * fps)
370             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only

```

```

cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])

```

```

451         work.append(g[split_i:])
452     else:
453         out.append(g)
454         continue
455         work.append(g[:best_i])
456         work.append(g[best_i:])
457     out.sort(key=lambda grp: grp[0][0])
458     return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
466         ``blob_factor`` x
467         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
468         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
469         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
470         (~1?2x the cap) is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
471         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
472         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
473         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
474         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
475         that piece came from a forced chop. Iterative worklist to bound stack depth."""
476         if max_win_frames <= 0:
477             return groups, [False] * len(groups)
478         blob_frames = max_win_frames * max(1.0, blob_factor)
479         out: List[List[Tuple[int, float]]] = []
480         flags: List[bool] = []
481         forced = 0
482         for g in groups:
483             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
484                 out.append(g)
485                 flags.append(False)
486                 continue
487             pieces: List[List[Tuple[int, float]]] = []
488             work = [g]
489             while work:
490                 h = work.pop()
491                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
492                     pieces.append(h)
493                     continue
494                     mid = (h[0][0] + h[-1][0]) / 2.0
495                     split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
496                     work.append(h[:split_i])
497                     work.append(h[split_i:])
498                     pieces.sort(key=lambda grp: grp[0][0])
499                     out.extend(pieces)
500                     flags.extend([True] * len(pieces))
501                     forced += len(pieces) - 1
502             if forced:
503                 logger.warning(
504                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
505                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /

```

```

two-sided "
505         "motion), not a bigger cap", forced)
506     return out, flags
507

```

13. assistant (2026-06-18T01:33:33.912Z)

Let me examine the splits, calibration, classifier, and other stats-heavy files for boundary conditions.

14. user (2026-06-18T01:33:34.878Z)

```

1     """Train/eval split guard (pure-logic core, in-container).
2
3     Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4     and plan M0:
5     - Split unit = match (never clip / file MD5).
6     - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7     - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8     - This would have caught train/eval leakage in training/eval paths.
9     - Cross-repo collector manifest `split` field wiring + wiring into
10       distill_local / run_regression deferred to later PR.
11
12     Usage (once wired):
13         train, eval_ = load_split(manifest)
14         assert_disjoint(train, eval_)
15     """
16     from __future__ import annotations
17
18     import json
19     import os
20     from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23     def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
24     Tuple[Set[str], Set[str]]:
25         """Load (train_ids, eval_ids) sets keyed by match id.
26
27         manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
28         have 'match_id'
29         (or 'id' fallback; must be non-empty str for real match key; name/video_id
30         deliberately not used as they are weak/MD5).
31
32         Raises ValueError on:
33         - non-list manifest after load
34         - no real match-level key (empty/falsy after explicit check)
35         - split label not exactly "train" or "eval"
36
37         The collector 'split' field and cross-repo wiring is deferred; this supports
38         synthetic or pre-split manifests for the pure-logic guard and tests.
39         """
40         if isinstance(manifest, (str, bytes, os.PathLike)):
41             with open(manifest, "r", encoding="utf-8-sig") as f:
42                 raw = json.load(f)
43                 if not isinstance(raw, list):
44                     raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
45                 data = cast(List[Dict[str, Any]], raw)
46         else:
47             if isinstance(manifest, dict):
48                 raise ValueError(f"manifest must be a list of entries, got
49 {type(manifest)}")
50             data = list(manifest)
51             if not isinstance(data, list):
52                 raise ValueError(f"manifest must be a list of entries, got {type(data)}")

```

```

50     train_ids: Set[str] = set()
51     eval_ids: Set[str] = set()
52
53     for entry in data:
54         if not isinstance(entry, dict):
55             continue
56         # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
57         mid = entry.get("match_id")
58         if mid is None or str(mid).strip() == "":
59             mid = entry.get("id")
60         if mid is None or str(mid).strip() == "":
61             raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62         mid = str(mid).strip()
63         sp = str(entry.get("split", "")).strip().lower()
64         if sp not in ("train", "eval"):
65             raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66         if sp == "eval":
67             eval_ids.add(mid)
68         else:
69             train_ids.add(mid)
70
71     return train_ids, eval_ids
72
73
74 def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75     """Raise if any match-id overlap between train and eval sets.
76
77     This would have caught train/eval leakage (e.g. a video used for both
78     training a segmenter and for regression/eval baselines).
79     """
80     t = set(str(x) for x in train_ids)
81     e = set(str(x) for x in eval_ids)
82     overlap = t & e
83     if overlap:
84         raise ValueError(
85             f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86             "This would have caught train/eval leakage."
87         )
88

```

15. user (2026-06-18T01:33:35.501Z)

```

1     """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3     This is the measurement layer for per-video / per-corpus calibration. It answers
4     two questions honestly:
5
6     1. """Did precision improve, by how much?""" ? `improvement_report` scores a
7         baseline config vs a tuned config on the same ground truth and reports the
8         before/after metrics and the % delta.
9
10    2. """Is that gain real or overfit?""" ? calibration must be measured on data the
11        tuning never saw, otherwise you're just memorising. Two protocols:
12        - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13            train folds, score on the held-out fold. The **generalization_gap**
14            (train ? test) is the overfit alarm.
15        - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16            pick the config on all *other* videos, score on the held-out video. Reports
17            per-video held-out scores + mean±std. This is the number to trust.
18
19    Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20    List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same

```

```

21     harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22     """
23     from __future__ import annotations
24
25     import statistics as _st
26     from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28     from backend.eval.metrics import score, Score, Interval
29
30     Config = Any                                # opaque to this module (e.g. a thresholds
dict)
31     PredictFn = Callable[[Config], List[Interval]]
32
33
34     def pct_improvement(before: float, after: float) -> float:
35         """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36         if before <= 0:
37             return float("inf") if after > 0 else 0.0
38         return (after - before) / before * 100.0
39
40
41     def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42         """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43         return score(preds, gts, iou_threshold)
44
45
46     def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47         """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48         if n <= 0:
49             return []
50         k = max(2, min(k, n))                    # need >=2 folds; cap at n
51         folds = [list(range(i, n, k)) for i in range(k)]    # round-robin ? balanced,
deterministic
52         splits = []
53         for f in range(k):
54             test = folds[f]
55             train = [i for i in range(n) if i not in set(test)]
56             if test and train:
57                 splits.append((train, test))
58         return splits
59
60
61     def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                     metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63         """Pick the config whose predictions maximise `metric` against `gts`."""
64         best_cfg, best_val = None, -1.0
65         for cfg in configs:
66             val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67             if val > best_val:
68                 best_cfg, best_val = cfg, val
69         return best_cfg, best_val
70
71
72     def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73                       k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74         """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76         Per fold: choose the best config on the train rallies, score it on the held-out
77         rallies. A large `generalization_gap` (train >> test) means the thresholds are
78         overfitting that video's quirks.
79
80         Default metric is **recall** on purpose: predictions are global (whole-video

```



```

81 windows), so scoring them against a *fold subset* of GT would unfairly count the
82 other rallies' windows as false positives ? precision/F1 are not well-defined
83 per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
84 is the honest within-video generalization signal. For precision/F1, use
85 `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86 held-out video.
87 """
88 splits = kfold_indices(len(gts), k)
89 if not splits:
90     return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91 train_scores, test_scores, picked = [], [], []
92 for train_idx, test_idx in splits:
93     gts_tr = [gts[i] for i in train_idx]
94     gts_te = [gts[i] for i in test_idx]
95     cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96     te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97     train_scores.append(tr)
98     test_scores.append(te)
99     picked.append(cfg)
100 mean_test = _st.mean(test_scores)
101 mean_train = _st.mean(train_scores)
102 return {
103     "metric": metric,
104     "folds": len(splits),
105     "mean_test": mean_test,
106     "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
107     "mean_train": mean_train,
108     "generalization_gap": mean_train - mean_test,    # >~0.1 = likely overfitting
109     "picked_configs": picked,
110 }
111
112
113 def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
114                         metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
115     """Leave-one-video-out CV ? the honest metric as the corpus grows.
116
117     `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
118     For each held-out video: pick the config that's best *pooled across the other
119     videos*, then score it on the held-out one. The mean held-out score is the
120     "precision on unseen videos" number; per-video spread shows stability.
121     """
122     if len(videos) < 2:
123         return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
124     per_video = []
125     for i, held in enumerate(videos):
126         others = [v for j, v in enumerate(videos) if j != i]
127
128         def pooled_score(cfg: Config, others: List[dict] = others) -> float:
129             vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
130                     for v in others]
131             return _st.mean(vals)
132
133         best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
134         held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
135         per_video.append({"video": held.get("name", f"video_{i}"),
136                          "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
137     held_scores = [p["held_out"] for p in per_video]
138     train_scores = [p["train_pooled"] for p in per_video]
139     return {
140         "metric": metric,

```

```

141         "mean_held_out": _st.mean(held_scores),
142         "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
143         "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
144         "per_video": per_video,
145     }
146
147
148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                             gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X% precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164

```

16. assistant (2026-06-18T01:33:42.631Z)

Let me examine the R4 inactivity trim more closely ? there's a subtle sliding-window bug candidate there. Let me also look at the classifier and tolerance_metrics percentile handling, plus eval_partial_labels.

Looking at `\_trim\_inactive\_tail` (lines 391-417): the sliding window condition is `while frames[lo] <= frames[i] - trail\_frames`. This means the window contains frames `f` where `f > frames[i] - trail\_frames`. Let me check whether `frames[lo]` could advance past `i`. When `lo == i`, `frames[i] <= frames[i] - trail\_frames` is `0 <= -trail\_frames` which is false (trail_frames > 0), so `lo` never passes `i`. Good. `win\_count = i - lo + 1 >= 1`. That's fine.

Let me look at more files ? the classifier, eval_partial_labels, and per_user_eval which often have statistical edge cases.

17. user (2026-06-18T01:33:43.994Z)

```

1     """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3     The offline learned segmenter (ADR-004) must run locally with no external API and
4     no heavy ML stack ? the project already depends on numpy, and sklearn is not
5     installed. This implements just enough: standardized features, balanced-class
6     gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7     model is a small, human-readable, version-controllable file.
8
9     It deliberately stays simple: the rally-window features are few and roughly
10    linearly separable (high player motion + sustained activity => real rally), so a
11    well-regularized linear model is a sensible, inspectable baseline. Swap in a
12    richer model later behind the same fit/predict_proba/save/load interface.
13    """
14
15    import json
16    from typing import List, Optional
17
18    import numpy as np
19

```

```

20
21 class LogisticRegression:
22     def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23         self.lr = lr
24         self.epochs = epochs
25         self.l2 = l2
26         self.w: Optional[np.ndarray] = None
27         self.b: float = 0.0
28         self.mean: Optional[np.ndarray] = None
29         self.std: Optional[np.ndarray] = None
30         self.feature_names: Optional[List[str]] = None
31
32     @staticmethod
33     def _sigmoid(z: np.ndarray) -> np.ndarray:
34         # Clip for numerical stability (avoid overflow in exp).
35         return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37     def _standardize(self, X: np.ndarray) -> np.ndarray:
38         return (X - self.mean) / self.std
39
40     def fit(self, X, y, feature_names: Optional[List[str]] = None,
41            class_weight: str = "balanced") -> "LogisticRegression":
42         """Fit with optional balanced class weighting (rally windows are imbalanced
43         ? many spurious candidates per real rally)."""
44         X = np.asarray(X, dtype=float)
45         y = np.asarray(y, dtype=float)
46         if X.ndim != 2 or X.shape[0] == 0:
47             raise ValueError("X must be a non-empty 2D array")
48         self.feature_names = feature_names
49
50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----

```

```

85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))
115

```

18. user (2026-06-18T01:33:44.262Z)

```

1     """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3     The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4     ending_reason,sport). While a video is only partially tagged, the labels cover a
5     PREFIX of the video ? every rally up to some point is marked, and everything after is
6     simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7     the detector's *correct* rallies in the un-labeled tail would be counted as false
8     positives and crush precision.
9
10    This harness fixes that by restricting the evaluation to the LABELED WINDOW
11    [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12    dropping predicted segments that fall outside it. Within that window the human labels
13    ARE complete, so precision/recall/F1 are honest.
14
15    It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16    - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17    - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18    - eval.metrics.match_intervals / score
19    - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
20
21    Run:
22    python -m backend.eval.eval_partial_labels \
23        --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24        --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
25    Avi.rallies.csv" \
26        --fps 59.94 --frame-width 1920
27
28    """
29    from __future__ import annotations
30
31    import argparse

```

```

30 import os.path as osp
31 from typing import List, Optional, Tuple
32
33 from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
34 from backend.eval.metrics import score
35 from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
36 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37 from backend.pipeline.detectors import rally_gate
38
39 Interval = Tuple[float, float]
40 Config = Tuple[float, float, float]
41
42 # Operating points to compare. The tuned config wins on its fit-video but over-segments
43 # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44 TUNED: Config = (0.05, 1.0, 1.0)
45
46
47 def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48     """Keep intervals that overlap the labeled window [w0, w1].
49
50     A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51     in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52     they are neither rewarded nor penalized. GTs are assumed already inside the window.
53     """
54     return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57 def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                 window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59     """Score one (config, gate) operating point inside the labeled window."""
60     preds = predict_fn(cfg)
61     if gate_crossings > 0 and points:
62         preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63     preds = restrict_to_window(preds, *window)
64     sc = score(preds, gts, iou)
65     return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
66 "preds": preds}
67
68
69 def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
70        window_start: float = 0.0, window_end: Optional[float] = None,
71        gate_crossings: int = 2, iou: float = 0.5,
72        comparison_out: Optional[str] = None) -> dict:
73     gts_all = load_golden_gts(labels_csv)
74     if not gts_all:
75         raise SystemExit(f"no usable human rallies in {labels_csv}")
76     if window_end is None:
77         window_end = max(e for _, e in gts_all)
78     window = (window_start, window_end)
79     gts = restrict_to_window(gts_all, *window)
80
81     predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
82     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
83     traj_end = (max(p.frame for p in points) / fps) if points else 0.0
84
85     print("=== Pipeline vs PARTIAL human labels ===")
86     print(f"labels: {labels_csv}")
87     print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
88 {window_end:.1f}s] "
89           f"-> {len(gts)} rallies in-window")
90     print(f"trajectory covers ~0-{traj_end:.1f}s "
91           f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
92 than window'})")
93     print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")

```

```

91
92     # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
93     points_for_gate = points
94     candidates = [
95         ("baseline", BASELINE, 0),
96         (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
97         ("tuned", TUNED, 0),
98         (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
99     ]
100     results = []
101     header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
102         f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
103     print(header)
104     print("-" * len(header))
105     for name, cfg, gate in candidates:
106         r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
107         sc = r["score"]
108         r["name"] = name
109         results.append(r)
110         print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
{sc.false_negatives:>3} "
111             f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
112             f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114     best = max(results, key=lambda r: r["score"].f1)
115     print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117     # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
118     comp = build_comparison(best["preds"], gts, iou)
119     comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120     write_comparison_csv(comp_out, comp)
121     print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123     return {
124         "window": window, "n_human_in_window": len(gts),
125         "results": [{ "name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                     "n_pred": r["n_pred"], **r["score"].as_dict() } for r in results],
127         "best": best["name"], "comparison_csv": comp_out,
128     }
129
130
131     # Grid centered on the two levers the human-GT failure analysis flagged:
132     # min_window_duration DOWN (recover short 3-5s rallies) and merge_gap UP (heal
133     # fragmented long rallies). Gate is swept on/off because it can over-prune.
134     DEFAULT_GRID = [
135         (vt, mg, mwd)
136         for vt in (0.01, 0.02, 0.03, 0.05)
137         for mg in (1.0, 2.0, 3.0, 4.0)
138         for mwd in (0.5, 1.0, 1.5, 2.0)
139     ]
140
141
142     def sweep(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
143             window_start: float = 0.0, window_end: Optional[float] = None,
144             gate_options: Tuple[int, ...] = (0, 2), iou: float = 0.5,
145             grid: Optional[List[Config]] = None, top: int = 12) -> dict:
146         """Grid-search (velocity, merge_gap, min_dur) x gate vs human labels, in-window.
147
148         WARNING: this is fit-on-self on ONE video -> optimistic. It shows the achievable
149         ceiling + which levers matter, NOT a transferable config. The honest operating
150         point comes from leave-one-video-out once >=3 videos are labeled.

```

```

151     """
152     grid = grid or DEFAULT_GRID
153     gts_all = load_golden_gts(labels_csv)
154     if not gts_all:
155         raise SystemExit(f"no usable human rallies in {labels_csv}")
156     if window_end is None:
157         window_end = max(e for _, e in gts_all)
158     window = (window_start, window_end)
159     gts = restrict_to_window(gts_all, *window)
160     predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
161     points = TrackNetRunner.parse_trajectory_csv(trajjectory_csv)
162
163     rows: List[dict] = []
164     for cfg in grid:
165         for gate in gate_options:
166             r = _eval_point(predict_fn, cfg, points, fps, gts, window, gate, iou)
167             sc = r["score"]
168             rows.append({"cfg": cfg, "gate": gate, "n_pred": r["n_pred"], "f1": sc.f1,
169                         "precision": sc.precision, "recall": sc.recall, "mean_iou":
170                         "tp": sc.true_positives, "fp": sc.false_positives, "fn":
171                         sc.false_negatives})
172
173     rows.sort(key=lambda x: (x["f1"], x["mean_iou"]), reverse=True)
174
175     print(f"=== Config sweep vs human labels ({len(gts)} rallies in "
176           f"{window_start:.0f},{window_end:.0f}]s; {len(grid)}x{len(gate_options)}
177           configs) ===")
178     print("NOTE: fit-on-self on ONE video = optimistic; real test is leave-one-video-
179           out.\n")
180     hdr = f"{'#':>3} {'veloc':>6} {'merge':>6} {'minDur':>6} {'gate':>4} " \
181           f"{'F1':>6} {'prec':>6} {'recall':>6} {'mIoU':>6} {'TP':>3} {'FP':>3}
182           {'FN':>3}"
183     print(hdr)
184     print("-" * len(hdr))
185     for i, r in enumerate(rows[:top], 1):
186         vt, mg, mwd = r["cfg"]
187         print(f"{i:>3} {vt:>6.3f} {mg:>6.1f} {mwd:>6.1f} {r['gate']:>4} "
188               f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f}
189               {r['mean_iou']:>6.3f} "
190               f"{r['tp']:>3} {r['fp']:>3} {r['fn']:>3}")
191     b = rows[0]
192     print(f"\nbest: velocity={b['cfg'][0]} merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]}
193           "
194           f"gate={b['gate']} -> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R
195           {b['recall']:.3f})")
196     return {"window": window, "n_gt": len(gts), "ranked": rows, "best": b}
197
198 def main() -> None:
199     ap = argparse.ArgumentParser(description=__doc__)
200     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
201                    (Frame,Visibility,X,Y)")
202     ap.add_argument("--labels", required=True, help="human golden CSV
203                    (rally_number,start_time,end_time,...)")
204     ap.add_argument("--fps", type=float, default=59.94)
205     ap.add_argument("--frame-width", type=float, default=1920.0)
206     ap.add_argument("--window-start", type=float, default=0.0)
207     ap.add_argument("--window-end", type=float, default=None,
208                     help="end of the labeled region (default: last labeled rally end)")
209     ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
210                    recommended)")
211     ap.add_argument("--iou", type=float, default=0.5)
212     ap.add_argument("--comparison-out", default=None)
213     ap.add_argument("--sweep", action="store_true",
214                     help="grid-search configs vs labels (fit-on-self; finds the ceiling

```

```

+ best levers)")
205     ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
206     args = ap.parse_args()
207     if args.sweep:
208         sweep(args.trajectory, args.labels, args.fps, args.frame_width,
209               window_start=args.window_start, window_end=args.window_end,
210               iou=args.iou, top=args.top)
211     else:
212         run(args.trajectory, args.labels, args.fps, args.frame_width,
213             window_start=args.window_start, window_end=args.window_end,
214             gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
215
216
217 if __name__ == "__main__":
218     main()
219

```

19. assistant (2026-06-18T01:33:47.218Z)

Now let me look at the rally_gate (net-crossing detection - frame/time windowing), per_user_eval, promotion, and harness files. Let me also check the bootstrap_ci percentile indexing more carefully against eval_stats.

20. user (2026-06-18T01:33:48.656Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool
35
36
37    def _spread(vals: Sequence[float]) -> float:
38        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39        if len(vals) < 2:
40            return 0.0

```



```

41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []

```

```

104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

21. user (2026-06-18T01:33:48.958Z)

```

1     """Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release
gate.
2
3     The personalization feature's promise is operational, not aspirational: *after the user
4     contributes K of their own videos, the personalized (candidate) recipe should beat the
5     generic (control) one on the user's STILL-UNSEEN footage.* This module turns that
promise
6     into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each
``k``,
7     **does the candidate beat control on a held-out split, honestly?**
8
9     It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math.
Every
10    fold reuses the shipped experiment primitives ?
11
12    - `experiment.predict` ? variant ? held-out intervals (static variants need no fit);
13    - `experiment._train` ? fit a learned variant on the corpus split (the same honest
14      train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
15    - `metrics.score` ? per-video held-out F1;
16    - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign
test, C1).
17
18    **Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first
``k``
19    videos are the *personalization corpus* (fit/operating-point source) and the remaining
20    `len(videos) - k` are *held-out* (scored, never seen during fit). This mirrors the
product
21    reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and
keeps
22    ``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.
23
24    The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
25    control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out
videos
26    than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a
bare
27    mean. ``None`` means the criterion was never met on this corpus (the honest negative).
28
29    Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
30    `VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` +
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
31    """
32    from __future__ import annotations
33
34    from typing import Any, Dict, List, Optional, Sequence
35
36    from backend.eval import eval_stats as _stats

```

```

37     from backend.eval import experiment
38     from backend.eval.classifier import LogisticRegression
39     from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
40     from backend.eval.metrics import score
41
42
43     def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
44                   iou: float) -> Optional[LogisticRegression]:
45         """The fitted model a variant needs to predict held-out videos, or None if it needs
46         none.
47         - **static variant** (no learned filter): None ? `predict` scores directly, no
48         leakage.
49         - **pinned model** (`classifier_model` set): the shipped model, loaded as-is (no
50         per-fold
51         training ? reports "how the shipped artifact does on this user's held-out
52         footage").
53         - **learned recipe** (`feature_names`, no pinned model): fit on the `corpus`
54         split via
55         the harness's own honest training step. The corpus and held-out sets are disjoint,
56         so the
57         held-out scores are genuinely out-of-sample.
58         """
59         if variant.classifier_model is not None:
60             return LogisticRegression.load(variant.classifier_model)
61         if variant.feature_names:
62             return experiment._train(variant, corpus, iou)
63         return None
64
65     def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
66                     heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
67         """Per-held-out-video F1 for `variant`, fit on `corpus` (if learned), scored on
68         `heldout`. Held-out videos are video-aligned with the returned list (caller pairs
69         B?A)."""
70         model = _fit_model(variant, corpus, iou)
71         f1s: List[float] = []
72         for vc in heldout:
73             assert vc.gts is not None # guarded in learning_curve before any scoring
74             preds = experiment.predict(variant, vc, model=model)
75             f1s.append(score(preds, vc.gts, iou).f1)
76         return f1s
77
78     def learning_curve(videos: Sequence[VideoCandidates],
79                       control_variant: Variant,
80                       candidate_variant: Variant,
81                       *,
82                       iou: float = 0.5,
83                       min_k: int = 2,
84                       seed: int = 0) -> Dict[str, Any]:
85         """Sweep corpus size `k` and report where the candidate first honestly beats
86         control.
87         For one user's `videos`, for each `k` in `min_k .. len(videos) - 1`: fit/score
88         both
89         variants on the first `k` videos (the personalization corpus) and evaluate on the
90         remaining
91         held-out videos. Per-held-out-video F1s are paired B?A and summarised with
92         `eval_stats.paired_delta_ci` (mean ?F1 + bootstrap CI + exact sign test).
93         Returns `{n_videos, min_k, iou, per_k, smallest_k_with_lift}` where each `per_k`
94         record is::
95
96             {k, n_heldout, control_f1, candidate_f1,

```

```

91         mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}
92
93     ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that
94     ``k``.
95     ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ?
96     candidate
97     above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly
98     more
99     held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).
100
101     Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap).
102     Raises
103     `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to
104     sweep).
105     """
106     n = len(videos)
107     for vc in videos:
108         if vc.gts is None:
109             raise ExperimentError(
110                 f"{vc.name} has no golden labels; learning_curve needs labeled held-out
111                 videos")
112     if min_k < 1:
113         raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
114     if n <= min_k:
115         # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out
116         video, so
117         # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest
118         curve.
119         return {
120             "n_videos": n,
121             "min_k": min_k,
122             "iou": iou,
123             "per_k": [],
124             "smallest_k_with_lift": None,
125         }
126
127     per_k: List[Dict[str, Any]] = []
128     smallest_k_with_lift: Optional[int] = None
129     for k in range(min_k, n):
130         corpus = videos[:k]
131         heldout = videos[k:]
132         control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
133         candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)
134
135         paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
136         sign_test = paired["sign_test"]
137         # Honest lift = the candidate is above control AND the CI clears 0 AND the sign
138         test
139         # agrees (B wins on strictly more held-out videos than it loses) ? the promotion
140         bar.
141         control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
142         candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else
143         0.0
144         has_lift = (
145             candidate_mean > control_mean
146             and bool(paired["ci_excludes_zero"])
147             and paired["mean_delta"] > 0.0
148             and sign_test["wins"] > sign_test["losses"]
149         )
150         per_k.append({
151             "k": k,
152             "n_heldout": len(heldout),
153             "control_f1": control_mean,
154             "candidate_f1": candidate_mean,
155             "mean_delta": paired["mean_delta"],

```

```

145         "ci_low": paired["ci_low"],
146         "ci_high": paired["ci_high"],
147         "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
148         "sign_test": sign_test,
149     })
150     if has_lift and smallest_k_with_lift is None:
151         smallest_k_with_lift = k
152
153     return {
154         "n_videos": n,
155         "min_k": min_k,
156         "iou": iou,
157         "per_k": per_k,
158         "smallest_k_with_lift": smallest_k_with_lift,
159     }
160

```

22. assistant (2026-06-18T01:33:54.368Z)

Now let me look carefully at the ``count_net_crossings`` function and ``_points_in``. There's a potential issue: ``_points_in`` uses ``start * fps <= p.frame <= end * fps`` ? this is frame index comparison. That seems consistent. Let me examine the `bootstrap_ci` percentile bounds more carefully, and look at `promotion.py`, `harness.py`, and the `calibrate` files.

Let me cross-cut on a specific concern in ``bootstrap_ci``: the percentile index computation. With ``confidence=0.95``, ``alpha=0.025``, ``last=n_boot-1``. ``lo = means[floor(0.025*last)]``, ``hi = means[ceil(0.975*last)]``. This looks reasonable.

Let me check the `harness`, `promotion`, and `gt_loader` files.

23. user (2026-06-18T01:33:55.767Z)

```

1     """Score the pipeline's detected rallies against ground-truth annotations.
2
3     Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4     so scoring is fast, deterministic, and free. Two prediction "stages" can be
5     scored:
6
7     - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8       the shuttle/person detector found, before AI confirmation.
9     - ``final`` ? confirmed play segments (``play_segments`` table). After the
10      AI handoff trimmed/validated boundaries.
11
12     Comparing the two tells you whether the weak link is *detection* (candidates
13     already miss rallies) or *segmentation* (candidates catch them but final is
14     wrong) ? see docs/EVALUATION.md.
15     """
16
17     import logging
18     from dataclasses import dataclass, field
19     from typing import Any, List, Dict, Optional
20
21     from backend.infrastructure.database import Database
22     from backend.eval import metrics
23     from backend.eval.metrics import Interval, Score, MatchResult
24
25     logger = logging.getLogger(__name__)
26
27     STAGES = ("candidates", "final")
28
29
30     @dataclass
31     class StageReport:
32         stage: str
33         pred_intervals: List[Interval]

```

```

34     score: Score
35     match_result: MatchResult
36     pr_curve: List[Score] = field(default_factory=list)
37
38
39 @dataclass
40 class EvalReport:
41     video_id: str
42     iou_threshold: float
43     gt_intervals: List[Interval]
44     stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47 def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48     """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49     ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50     ivs.sort(key=lambda iv: iv[0])
51     return ivs
52
53
54 def get_predictions(db: Database, video_id: str, stage: str,
55                    detector: Optional[str] = None) -> List[Interval]:
56     """Fetch predicted rally intervals for one stage from the DB."""
57     if stage == "candidates":
58         rows = db.query_candidate_segments(video_id, detector=detector)
59     elif stage == "final":
60         rows = db.query_play_segments(video_id, {})
61     else:
62         raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63     return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67             stages: List[str], iou_threshold: float = 0.5,
68             detector: Optional[str] = None,
69             with_pr_curve: bool = False) -> EvalReport:
70     """Score the requested stages for one video against ground truth."""
71     report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                         gt_intervals=gt_intervals)
73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86     seconds = max(0, int(seconds))
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93     """JSON-serialisable view of an EvalReport."""
94     out: Dict[str, Any] = {
95         "video_id": report.video_id,
96         "iou_threshold": report.iou_threshold,
97         "num_ground_truth": len(report.gt_intervals),
98         "stages": {},

```

```

99         }
100     for stage, sr in report.stages.items():
101         out["stages"][stage] = {
102             "score": sr.score.as_dict(),
103             "pr_curve": [s.as_dict() for s in sr.pr_curve],
104         }
105     return out
106
107
108 def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109     """Human-readable console report for one video."""
110     lines = [
111         "=" * 64,
112         f" Rally evaluation ? video '{report.video_id}'",
113         f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114         "=" * 64,
115         f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116         f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117         "-" * 64,
118     ]
119     for stage in report.stages:
120         s = report.stages[stage].score
121         lines.append(
122             f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123             f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124             f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125         )
126     lines.append("=" * 64)
127
128     if show_failures:
129         for stage, sr in report.stages.items():
130             misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131             fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132             lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133             for iv in misses:
134                 lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135             lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136             for iv in fps:
137                 lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139     return "\n".join(lines)
140

```

24. user (2026-06-18T01:33:56.155Z)

```

1     """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2     at the SERVED windowing.
3
4     **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5     honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6     it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7     PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8     cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9     regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the

```

```

10     served windowing** is.
11
12     This module composes ? it adds no new statistics. It layers ONE rule on top of
13     ``should_promote_for_user``:
14
15         A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16         REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17         estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19     A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20     can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21     verdicts so the eval?serve contrast is explicit and auditable.
22
23     Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24     paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25     """
26     from __future__ import annotations
27
28     from dataclasses import dataclass, field
29     from typing import Any, Dict, Optional, Sequence
30
31     from backend.eval.eval_stats import paired_delta_ci
32     from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34     __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37     def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                          eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39         """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41         ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42         (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43         a single ``regresses`` boolean:
44
45             - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46             the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47             - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48             ``-eps`` and the CI is not a confident net-negative).
49
50             ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51             positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52             """
53             delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54             mean_delta = float(delta["mean_delta"])
55             ci_low = float(delta["ci_low"])
56             ci_high = float(delta["ci_high"])
57             # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58             mean_regresses = mean_delta < -eps

```



```

59         ci_confidently_negative = ci_high < -eps
60         regresses = bool(mean_regresses or ci_confidently_negative)
61         return {
62             "regresses": regresses,
63             "mean_delta": mean_delta,
64             "ci_low": ci_low,
65             "ci_high": ci_high,
66             "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67             "sign_test": delta["sign_test"],
68             "n": int(delta["n"]),
69             "eps": float(eps),
70         }
71
72
73     @dataclass
74     class ServedGateResult:
75         """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76         proposal-windowing view, so the eval?serve contrast is never hidden.
77
78         - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79         *and* the served check did not veto (the cue does not regress at the served
windowing).
80         - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
81         - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82         served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83         - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84         *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85         - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86         tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87         - ``reason`` ? short human-readable explanation of the final verdict.
88         """
89
90         promote: bool
91         served_safe: bool
92         vetoed_by_served: bool
93         reason: str
94         base_decision: PromotionDecision
95         served: Dict[str, Any]
96         proposal: Optional[Dict[str, Any]] = field(default=None)
97
98         def as_dict(self) -> Dict[str, Any]:
99             return {
100                 "promote": self.promote,
101                 "served_safe": self.served_safe,
102                 "vetoed_by_served": self.vetoed_by_served,
103                 "reason": self.reason,
104                 "base_decision": self.base_decision.as_dict(),
105                 "served": self.served,
106                 "proposal": self.proposal,
107             }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],
112         served_b: Sequence[float],
113         *,
114         proposal_a: Optional[Sequence[float]] = None,
115         proposal_b: Optional[Sequence[float]] = None,

```

```

116     structural: bool = False,
117     min_n: int = 5,
118     p_threshold: float = 0.05,
119     served_regress_eps: float = 0.0,
120     n_boot: int = 2000,
121     seed: int = 0,
122 ) -> ServedGateResult:
123     """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
124
125     The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign
test) is
126     applied to the **served** scores ? the operating point production runs ? so a
proposal-only
127     win can never carry a promotion. The optional `proposal_` scores are scored too
and
128     reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
129     own).
130
131     Args:
132         served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133         measured at the **SERVED** windowing. These drive the decision.
134         proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
135         contrast; the served regression guard still has the final say.
136         structural: forwarded to `should_promote_for_user` ? a structural guard (e.g.
Gate-A vs
137         held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138         the served default if it regresses served. So for `structural=True` we report
139         `keep_on` honestly but `promote` reflects the served-safety check.
140         served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142     Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143     """
144     # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
145     base = should_promote_for_user(
146         served_a, served_b,
147         structural=structural, min_n=min_n, p_threshold=p_threshold,
148         n_boot=n_boot, seed=seed,
149     )
150
151     # 2) The eval?serve guard: does B regress A at the served windowing?
152     served = served_regresses(served_a, served_b, eps=served_regress_eps,
153                             n_boot=n_boot, seed=seed)
154     served_safe = not served["regresses"]
155
156     # 3) Optional proposal-windowing view (reported, never promoting on its own).
157     proposal: Optional[Dict[str, Any]] = None
158     if proposal_a is not None and proposal_b is not None:
159         proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
160                                     n_boot=n_boot, seed=seed)
161
162     # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
163     if structural:
164         # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
165         # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
166         # served, promote=False (don't bake an eval-only win into production) while

```

```

keep_on holds.
167         promote = served_safe
168         vetoed = False
169         if served_safe:
170             reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                         f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172         else:
173             reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                         f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                         "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                         f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f})"
183                         f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
184                         "windowing production doesn't serve cannot promote.")
185             elif would_promote:
186                 promote = True
187                 vetoed = False
188                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                         f"{served['mean_delta']:+.4f}). " + base.reason)
190             else:
191                 promote = False
192                 vetoed = False
193                 reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204

```

25. assistant (2026-06-18T01:33:59.700Z)

Let me look at the per_user_gate, gt_loader, and the rally_seg_eval files, plus the calibrate_wasb to understand gt loading and the windowing for time/frame rounding.

26. user (2026-06-18T01:34:01.135Z)

```

1      """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3      The owner adopted personalization as a flag-gated feature on a generalization-
first
4      baseline: a cue/variant B may be flipped ON for one user only when that user's own
5      held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6      fall back to the shipped baseline (variant A). This module is the decision logic for
that
7      flip; it is pure (no I/O) and reuses ``backend.eval.eval_stats`` for every statistic
8      (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.

```

```

9
10     The counter-case this guards against (and its mitigations, implemented here):
11
12     1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15     2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18     3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24     Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25     default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26     threaded through). Nothing here changes existing eval behaviour; callers opt in.
27     """
28     from __future__ import annotations
29
30     from dataclasses import dataclass, field
31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43         cue that cleared the full bar).
44         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46         ``promote``.
47         - ``reason`` ? short human-readable explanation of the decision.
48         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49         so a caller can log/audit exactly why a user was or wasn't personalized.
50         """
51
52         promote: bool
53         keep_on: bool
54         reason: str
55         n: int
56         mean_delta: float
57         ci_low: float
58         ci_high: float
59         ci_excludes_zero: bool
60         sign_test: Dict[str, float] = field(default_factory=dict)

```

```

61     floor_met: bool = False
62     structural_protected: bool = False
63
64     def as_dict(self) -> Dict[str, Any]:
65         """Plain-dict view (for JSON logging / telemetry)."""
66         return {
67             "promote": self.promote,
68             "keep_on": self.keep_on,
69             "reason": self.reason,
70             "n": self.n,
71             "mean_delta": self.mean_delta,
72             "ci_low": self.ci_low,
73             "ci_high": self.ci_high,
74             "ci_excludes_zero": self.ci_excludes_zero,
75             "sign_test": self.sign_test,
76             "floor_met": self.floor_met,
77             "structural_protected": self.structural_protected,
78         }
79
80
81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95
96         ``a_scores`` / ``b_scores`` are the user's per-video held-out F1 (or any score)
97         for
98         variant A (control / shipped baseline) and variant B (with the cue), aligned by
99         video
100         (same order). The decision rules (faithful to the module docstring):
101
102         1. ``structural=True`` ? the guard is never disabled on "no lift": return
103             ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
104             value is
105             in failure modes the user's corpus may not yet contain.
106         2. Otherwise promote iff ALL hold ? the minimum-N floor (``n >= min_n``), the
107             paired
108             ?F1 bootstrap CI excludes 0, and the paired sign test agrees (majority of
109             videos
110             win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
111
112             Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
113             ``paired_sign_test``);
114             deterministic given ``seed``. Returns a :class:`PromotionDecision`.
115             """
116             n = min(len(a_scores), len(b_scores))
117             floor_met = n >= min_n
118
119             # --- Evidence (reuse eval_stats; never reinvent the statistics)
120             -----
121             if n == 0:
122                 # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
123                 sign_test: Dict[str, float] = paired_sign_test([], eps)
124                 mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
125             else:

```

```

118         delta = paired_delta_ci(
119             a_scores, b_scores,
120             confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121         )
122         mean_delta = float(delta["mean_delta"])
123         ci_low = float(delta["ci_low"])
124         ci_high = float(delta["ci_high"])
125         ci_excludes_zero = bool(delta["ci_excludes_zero"])
126         sign_test = delta["sign_test"]
127
128         sign_p = float(sign_test.get("p_value", 1.0))
129         sign_wins = float(sign_test.get("wins", 0.0))
130         sign_losses = float(sign_test.get("losses", 0.0))
131         # The sign test "agrees" with a promotion only if it is significant AND the majority
132         # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133         # never satisfy the promotion bar.
134         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136         # --- Rule 1: structural guards are never disabled on "no lift"
-----
137         if structural:
138             return PromotionDecision(
139                 promote=False,          # nothing to "promote" ? it ships ON by design
140                 keep_on=True,           # ALWAYS kept ON, regardless of measured per-user
lift
141                 reason=(
142                     "structural guard kept ON regardless of measured per-user lift "
143                     "(value is in failure modes this user's corpus may not yet contain)"
144                 ),
145                 n=n,
146                 mean_delta=mean_delta,
147                 ci_low=ci_low,
148                 ci_high=ci_high,
149                 ci_excludes_zero=ci_excludes_zero,
150                 sign_test=sign_test,
151                 floor_met=floor_met,
152                 structural_protected=True,
153             )
154
155         # --- Rule 2: minimum-N floor (refuse before even looking at lift)
-----
156         if not floor_met:
157             return PromotionDecision(
158                 promote=False,
159                 keep_on=False,          # fall back to the shipped baseline (variant A)
160                 reason=(
161                     f"below minimum-N floor (n={n} < min_n={min_n}); "
162                     "falling back to shipped baseline"
163                 ),
164                 n=n,
165                 mean_delta=mean_delta,
166                 ci_low=ci_low,
167                 ci_high=ci_high,
168                 ci_excludes_zero=ci_excludes_zero,
169                 sign_test=sign_test,
170                 floor_met=False,
171                 structural_protected=False,
172             )
173
174         # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
-----
175         if ci_excludes_zero and sign_agrees:
176             promote = True
177             reason = (

```

```

178         f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179         f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180         f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181     )
182     else:
183         promote = False
184         misses = []
185         if not ci_excludes_zero:
186             misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187         if not sign_agrees:
188             misses.append(
189                 f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
190                 f"p={sign_p:.4f}")
191             )
192         reason = (
193             "insufficient evidence (" + "; ".join(misses) + "); "
194             "falling back to shipped baseline"
195         )
196
197     return PromotionDecision(
198         promote=promote,
199         keep_on=promote,          # non-structural: keep_on tracks promote exactly
200         reason=reason,
201         n=n,
202         mean_delta=mean_delta,
203         ci_low=ci_low,
204         ci_high=ci_high,
205         ci_excludes_zero=ci_excludes_zero,
206         sign_test=sign_test,
207         floor_met=True,
208         structural_protected=False,
209     )
210

```

27. user (2026-06-18T01:34:01.355Z)

```

1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3      The repo historically carried two incompatible loaders:
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5        ``curate export`` format), raises on bad rows;
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12     flows through one reader. Both legacy entry points now delegate here.
13
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16       columns ? rally_number, ending_reason, sport, ? ? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20     """
21     import csv
22     import logging
23     from typing import List, Optional, Tuple
24
25     from backend.eval.annotations import parse_timestamp
26
27     logger = logging.getLogger(__name__)

```

```

27
28     Interval = Tuple[float, float]
29
30     _START_NAMES = ("start", "start_time")
31     _END_NAMES = ("end", "end_time")
32
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
None."""
36         cells = [(c or "").strip().lower() for c in row]
37         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39         if start_idx is not None and end_idx is not None:
40             return start_idx, end_idx
41         return None
42
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flatters precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                     continue
64                 if cols is None and not saw_header:
65                     header = _header_columns(row)
66                     if header is not None:
67                         cols = header
68                         saw_header = True
69                         continue    # consume the header row
70                 cols = (0, 1)    # headerless: positional start,end
71                 assert cols is not None    # every branch above assigns it
72                 si, ei = cols
73                 try:
74                     if len(row) <= max(si, ei):
75                         raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                     start = parse_timestamp(row[si])
77                     end = parse_timestamp(row[ei])
78                     if start < 0:
79                         raise ValueError(f"negative start ({start})")
80                     if start >= end:
81                         raise ValueError(f"start ({start}) >= end ({end})")
82                 except ValueError as e:
83                     if strict:
84                         raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                     skipped += 1
86                     continue
87                 intervals.append((start, end))
88
89         if skipped:

```



```

90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

28. assistant (2026-06-18T01:34:06.023Z)

Now let me examine `parse\_timestamp` in annotations.py (timestamp parsing ? common off-by-one / rounding source), and rally_seg_eval.py which aggregates per-video metrics. Let me also look at experiment.py for the variant prediction logic.

29. user (2026-06-18T01:34:07.434Z)

```

1     """Load hand-annotated ground-truth rally timestamps from CSV.
2
3     Annotation file format (one file per video):
4
5         start,end
6         00:01:12,00:01:39
7         102.5,131.0
8
9     - A header row `start,end` is optional (auto-detected and skipped).
10    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12    - Blank lines and `#` comment lines are ignored.
13    """
14
15    import csv
16    import logging
17    from typing import List, Tuple
18
19    logger = logging.getLogger(__name__)
20
21    Interval = Tuple[float, float]
22
23
24    def parse_timestamp(value: str) -> float:
25        """Parse a timestamp string to float seconds.
26
27        Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28        Raises ValueError on anything unparseable so bad annotations fail loudly.
29        """
30        value = value.strip()
31        if not value:
32            raise ValueError("empty timestamp")
33        if ":" in value:
34            parts = value.split(":")
35            if len(parts) > 3:
36                raise ValueError(f"too many ':' segments in timestamp '{value}'")
37            total = 0.0
38            for part in parts:
39                total = total * 60 + float(part)
40            return total
41        return float(value)
42
43
44    def load_annotations(csv_path: str) -> List[Interval]:
45        """Load and validate ground-truth rally intervals from a CSV file.

```

```

46
47     Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
48     golden conventions: positional/headered ``start,end`` (the collector export) and the
49     rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50     ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51     surface immediately rather than silently skewing scores. Returns intervals sorted by
52     start time.
53     """
54     from backend.eval.gt_loader import load_gt_intervals
55     intervals = load_gt_intervals(csv_path, strict=True)
56     logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57     return intervals
58
59
60     def write_scaffold(csv_path: str) -> None:
61         """Write an empty annotation file with just the header, for hand-filling."""
62         with open(csv_path, "w", newline="") as f:
63             writer = csv.writer(f)
64             writer.writerow(["start", "end"])
65

```

30. user (2026-06-18T01:34:07.838Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2     over-segmentation-sensitive metric set, honestly.
3
4     This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 /
§7)
5     measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6     IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7     segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9     For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11    operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13    diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15    GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16    trajectory path + fps + frame_width; the GT rally intervals come from
17    ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20    """
21    from __future__ import annotations
22
23    import argparse
24    import json
25    import os
26    from typing import Any, Dict, List, Optional
27
28    from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29    from backend.eval.metrics import Interval
30    from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31

```

```

32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36
37     # ----- #
38     # Pure scoring core (no I/O ? unit-testable)
39     # ----- #
40     def score_intervals(preds: List[Interval], gts: List[Interval],
41                        iou: float = DEFAULT_IOU,
42                        tau_start: Optional[float] = None,
43                        tau_end: Optional[float] = None) -> Dict[str, Any]:
44         """The full per-video metric row for predicted vs ground-truth rally intervals.
45
46         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
47         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
48         merged mega-window is *visible* even when IoU-F1 isn't moved.
49
50         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
going forward;
51         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
52         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
precision) + the
53         start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
54         """
55         from backend.eval import tolerance_metrics as tm
56         ts = tm.TAU_START if tau_start is None else tau_start
57         te = tm.TAU_END if tau_end is None else tau_end
58         sc = metrics.score(preds, gts, iou_threshold=iou)
59         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
60         ms = segmentation_metrics.merge_split_report(preds, gts)
61         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
62         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
63         osr = tm.over_seg_report(preds, gts)
64         return {
65             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
66             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
67             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
68             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
69             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
70             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
71             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
72             # --- R2 (tolerance-match) ? the task-faithful headline block ---
73             "tau_start": ts, "tau_end": te,
74             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
75             "tol_matched": float(len(tol_matches)),
76             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
77             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
78         }
79
80
81     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                 "precision", "recall",
83                 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
85
86     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:

```

```

87         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90         leave-one-video-out discipline (windows within a video are correlated)."""
91         out: Dict[str, Any] = {"n": len(rows)}
92         for k in _AGG_KEYS:
93             vals = [r[k] for r in rows]
94             out[k] = eval_stats.mean_with_ci(vals) if vals else None
95         return out
96
97
98         # ----- #
99         # Golden corpus I/O
100        # ----- #
101        def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                        features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103            """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104            intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged."""
105            with open(manifest_path) as fh:
106                manifest = json.load(fh)
107            out: List[Dict[str, Any]] = []
108            for v in manifest:
109                feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110                gts: List[Interval] = []
111                if os.path.exists(feat):
112                    with open(feat) as fh:
113                        gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
114                out.append({"name": v["name"], "trajectory": v["trajectory"],
115                           "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
116                           "gts": gts})
117            return out
118
119        def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                        min_crossings: int = 0) -> List[Interval]:
121            """Parse a video's trajectory, window it under `preset` ? predicted rally
intervals.
122
123            `min_crossings` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124            `rally_gate.gate_windows`): drop windows whose shuttle crosses the net fewer than
this many
125            times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126            from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127            windowing (W1) leaves behind.
128
129            NEVER-EMPTY safeguard (mirrors `FusionSegmenter._select_for_handoff`): if the gate
would
130            drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131            strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132
133            """
134            from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
135            points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
136            wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
137            ivs = windowing.windows_to_intervals(wins)
138            if min_crossings > 0 and ivs:
139                from backend.pipeline.detectors.rally_gate import gate_windows

```

```

139         gated = [(g.start, g.end)
140                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                     if g.kept]
142         # Gating emptied a non-empty video ? keep the ungated windows.
143         ivs = gated if gated else ivs
144     return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                 "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175         alongside so the refinement is auditable."""
176         from backend.eval import tolerance_metrics as tm
177         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179         by_name_b = {r["name"]: r for r in b["rows"]}
180         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181         delta: Dict[str, Any] = {}
182         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                                    [rc[k] for _, rc in paired])
185         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186         if guard_weight_merge is not None:
187             gkw["weight_merge"] = guard_weight_merge
188         if guard_weight_split is not None:
189             gkw["weight_split"] = guard_weight_split
190         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193

```

```

194 # ----- #
195 # Reporting
196 # ----- #
197 def _ci(m: Optional[Dict[str, Any]]) -> str:
198     return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f}, {m['ci_high']:.3f}]"
199
200
201 def _format_guard(g: Dict[str, Any]) -> str:
202     """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
203     basis = "rate" if g["use_rates"] else "count"
204     lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
205             f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
206             f"NET verdict: {'PASS' if g['passes'] else 'BLOCK'} "
207             f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
208             f"no_merge_rate_regress={g['no_merge_rate_regress']})",
209             f"strict per-video rule (R2 §2.3.3, for contrast): "
210             f"{'pass' if g['strict_passes'] else 'BLOCK'} "
211             f"(count increases ? merges:{len(g['new_merges'])})
splits:{len(g['new_splits'])})"]
212     if g["merge_rate_regress"]:
213         names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
214                             for r in g["merge_rate_regress"])
215         lines.append(f" ? merge_rate REGRESSED on: {names}")
216     return "\n".join(lines)
217
218
219 def format_report(result: Dict[str, Any]) -> str:
220     p = result["preset"]
221     bounded = []
222     if p.get("max_window_duration"):
223         bounded.append(f"cap={p['max_window_duration']} + ((hard) if
p.get("hard_cap") else ""))
224     if p.get("inactivity_end"):
225         bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}
226             f"/md={p['inactivity_min_density']}")
227     if p.get("blob_fallback"):
228         bounded.append("blob_fallback")
229     extra = (" " + ", ".join(bounded)) if bounded else ""
230     lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
231             f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
232             f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}", ""]
233     lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
234     lines.append("|---|---|---|---|---|---|---|")
235     for r in result["rows"]:
236         lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
237             f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
238             f"{r['num_pred']}/{r['num_gt']} |")
239     a = result["aggregate"]
240     lines += [f"***Aggregate (n={a['n']}, mean [95% CI]):** "
241             f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
{_ci(a['f1@0.5'])} · "
242             f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
243     # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
244     rows = result["rows"]
245     if rows and "tol_f1" in rows[0]:
246         ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
247         lines += [f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict

```

```

1:1)", "",
248         "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
249         "|---|---|---|---|---|---|---|---|")
250     for r in rows:
251         lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
252         f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
253         f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
254         lines += [f"***R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} . "
255         f"tolP {_ci(a['tol_precision'])} . tolR {_ci(a['tol_recall'])} . "
256         f"|?start| {_ci(a['dstart_abs_median'])} . |?end|
{_ci(a['dend_abs_median'])} "
257         f"(start?solved / end=the gap)"]
258     return "\n".join(lines)
259
260
261 def _resolve_preset(name: str) -> WindowingPreset:
262     if name in PRESETS:
263         return PRESETS[name]
264     raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
265
266
267 def main() -> None:
268     ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
269     ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
270     ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
271     ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
272     ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
273     ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
274     ap.add_argument("--min-crossings", type=int, default=0,
275         help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
276     ap.add_argument("--vs-min-crossings", type=int, default=0,
277         help="net-crossings gate on --vs (default: same as --min-
crossings)")
278     ap.add_argument("--max-window-duration", type=float, default=None,
279         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
280     ap.add_argument("--vs-max-window-duration", type=float, default=None,
281         help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
282         "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
283     ap.add_argument("--hard-cap", action="store_true",
284         help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
285         "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
286     ap.add_argument("--vs-hard-cap", action="store_true",
287         help="hard-cap fallback on --vs (default: same as --hard-cap)")
288     ap.add_argument("--blob-fallback", action="store_true",
289         help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
290         "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
291     ap.add_argument("--vs-blob-fallback", action="store_true",
292         help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
293     ap.add_argument("--blob-factor", type=float, default=None,
294         help="R5 magnitude gate (both presets): chop only groups > this x
cap (default 4.0)")
295     ap.add_argument("--inactivity-end", type=float, default=None,
296         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims

```

```

post-rally drift tail)")
297         ap.add_argument("--vs-inactivity-end", type=float, default=None,
298                         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
299         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
300                         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
301         ap.add_argument("--inactivity-min-density", type=float, default=None,
302                         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
303         ap.add_argument("--tau-start", type=float, default=None,
304                         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
305         ap.add_argument("--tau-end", type=float, default=None,
306                         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
307         ap.add_argument("--guard-weight-merge", type=float, default=None,
308                         help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
309         ap.add_argument("--guard-weight-split", type=float, default=None,
310                         help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
311         ap.add_argument("--guard-counts", action="store_true",
312                         help="R1 over-seg guard scores raw counts instead of normalized
rates "
313                             "(rates are the default ? comparable across a structural
mega?bounded change)")
314         ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
315         args = ap.parse_args()
316
317         from dataclasses import replace
318         golden = load_golden(args.manifest, args.features_dir)
319         def _apply_inactivity(preset, end_val):
320             if end_val is not None:
321                 preset = replace(preset, inactivity_end=end_val)
322             if args.inactivity_rally_thresh is not None:
323                 preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
324             if args.inactivity_min_density is not None:
325                 preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
326             return preset
327
328         base = _resolve_preset(args.preset)
329         if args.max_window_duration is not None:
330             base = replace(base, max_window_duration=args.max_window_duration)
331         if args.hard_cap:
332             base = replace(base, hard_cap=True)
333         if args.blob_fallback:
334             base = replace(base, blob_fallback=True)
335         if args.blob_factor is not None:
336             base = replace(base, blob_factor=args.blob_factor)
337         base = _apply_inactivity(base, args.inactivity_end)
338         if args.vs:
339             vs_mc = args.vs_min_crossings or args.min_crossings
340             cand = _resolve_preset(args.vs)
341             cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not
None
342                         else args.max_window_duration)
343             if cand_cap is not None:
344                 cand = replace(cand, max_window_duration=cand_cap)
345             if args.vs_hard_cap or args.hard_cap:
346                 cand = replace(cand, hard_cap=True)
347             if args.vs_blob_fallback or args.blob_fallback:
348                 cand = replace(cand, blob_fallback=True)

```



```

349         if args.blob_factor is not None:
350             cand = replace(cand, blob_factor=args.blob_factor)
351         cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end
is not None
352                                 else args.inactivity_end)
353         result = compare(golden, base, cand, args.iou,
354                         base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
355                         tau_start=args.tau_start, tau_end=args.tau_end,
356                         guard_weight_merge=args.guard_weight_merge,
357                         guard_weight_split=args.guard_weight_split,
358                         guard_use_rates=not args.guard_counts)
359         print(format_report(result["base"]))
360         print("\n" + format_report(result["cand"]))
361         d = result["delta"]
362         print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
363               f"n_paired={result['n_paired']}")
364         for k, dd in d.items():
365             st = dd["sign_test"]
366             print(f" ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
367                   f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
368             print(_format_guard(result["over_seg_guard"]))
369         else:
370             result = evaluate(golden, base, args.iou, args.min_crossings,
371                              tau_start=args.tau_start, tau_end=args.tau_end)
372             print(format_report(result))
373         if args.json_out:
374             with open(args.json_out, "w", encoding="utf-8") as fh:
375                 json.dump(result, fh, indent=2)
376             print(f"\n[written {args.json_out}]")
377
378
379     if __name__ == "__main__":
380         main()
381

```

31. assistant (2026-06-18T01:34:14.115Z)

Now let me look carefully at a few things I've flagged. First, let me re-examine the `\_points\_in` function in rally_gate.py. The window selection is `start \* fps <= p.frame <= end \* fps`. But in `trajectory\_to\_action\_windows`, the window start/end are computed as `group[0][0] / fps + chunk\_start`. So when `chunk\_start=0`, `start = frame/fps`, and `start \* fps = frame`. That round-trips fine.

Let me look at the `\_percentile` in tolerance_metrics and `\_spread` in rally_gate for off-by-one edge cases, and also examine the experiment.py and calibrate_wasb. Let me also check the `bootstrap\_ci` boundary when `confidence` makes `int(ceil(0.975\*last))` potentially equal to last.

Let me look at experiment.py and the remaining calibration/calibrate_wasb files.

32. user (2026-06-18T01:34:15.436Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)

```

```

11     - `backend.eval.classifier`      ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine`   ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`      ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`     ? `pct_improvement`
15
16 The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17 (per-frame signals ? run once on the GPU/WSL box, persisted into
18 `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
56 ratio
57 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
58 scores
59
60 logger = logging.getLogger(__name__)
61
62 # Where a comparison run appends one reproducible record. Tracked location (NOT under
63 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64 # regression.DEFAULT_BASELINE_PATH.
65 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66 _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68 class ExperimentError(ValueError):
69     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70     score an unlabeled video with no pinned model, or a gate referencing an absent
71     feature)."""
72
73 # ----- Variant

```

```

74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set
101       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102       (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,

```

```

134         "min_players": self.min_players,
135         "classifier_model": self.classifier_model,
136         "feature_names": self.feature_names,
137         "threshold": self.threshold,
138         "merge_gap": self.merge_gap,
139         "tune_threshold": self.tune_threshold,
140         "calibrate": self.calibrate,
141     }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                             name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
196     List[List[float]]:
197         cols = [_feature_column(vc, name) for name in feature_names]

```

```

197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237             variant.feature_names))]
238             if calibrator is not None:
239                 proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243             return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                             train_videos: Sequence[VideoCandidates], iou: float
247                             ) -> "tuple[Optional[Callable[[float], float]],
248                             Optional[float]]":
249         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
250         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
251         (use the variant default). Honest: never looks at the held-out video.
252
253         """
254         calibrator: Optional[Callable[[float], float]] = None
255         if variant.calibrate:
256             raw: List[float] = []
257             y: List[int] = []
258             for vc in train_videos:
259                 raw.extend(float(p) for p in
260                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
261                             [])))

```

```

257         y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258     if variant.calibrate == "platt":
259         calibrator = fit_platt(raw, y)
260     elif variant.calibrate == "isotonic":
261         calibrator = fit_isotonic(raw, y)
262     else:
263         raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264     threshold: Optional[float] = None
265     if variant.tune_threshold:
266         best_t, best_f1 = variant.threshold, -1.0
267         for k in range(1, 20):                                # grid 0.05..0.95 on the train
fold's rally F1
268             t = round(0.05 * k, 2)
269             f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                         vc.gts or [], iou).f1 for vc in train_videos]
271             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272             if mean_f1 > best_f1:
273                 best_f1, best_t = mean_f1, t
274             threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
SHIPPED model. ? optimistic if those videos were in its training set ? use this
305         to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310
311         """
312         for vc in videos:
313             if vc.gts is None:

```

```

313         raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315     per_video: List[Dict[str, Any]] = []
316     predictions: Dict[str, List[Interval]] = {}
317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319     pinned_model = (LogisticRegression.load(variant.classifier_model)
320                     if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                  for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}

```

```

367 overlaps = _segm.DEFAULT_OVERLAPS
368 f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369 ratios: List[float] = []
370 for name, preds in eval_v.get("predictions", {}).items():
371     gts = gts_by_name.get(name, [])
372     got = _segm.f1_at_overlaps(preds, gts, overlaps)
373     for ov in overlaps:
374         f1k[ov].append(got[ov])
375     ratios.append(_segm.segment_count_ratio(preds, gts))
376
377 def _mean(xs: List[float]) -> float:
378     return sum(xs) / len(xs) if xs else 0.0
379 out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380 out["segment_count_ratio"] = _mean(ratios)
381 return out
382
383
384 def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                   eval_b: Dict[str, Any]) -> Dict[str, Any]:
386     """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388     ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389     so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390     the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391     """
392     a_f1 = [p["f1"] for p in eval_a["per_video"]]
393     b_f1 = [p["f1"] for p in eval_b["per_video"]]
394     return {
395         "variant_a_ci": _ci_block(eval_a),
396         "variant_b_ci": _ci_block(eval_b),
397         "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398         "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                         "variant_b": _segmentation_block(videos, eval_b)},
400     }
401
402
403 def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404            iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405     """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406     golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408     The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415     shape. This is the measurement-honesty wiring (ANALYZERS_AND_RECONCILERS.md
§13/C1+C4): a
416     bare LOVO mean at n?6 is not trustworthy on its own.
417     """
418     if len(videos) < 1:

```



```

419         raise ExperimentError("compare needs at least one labeled video")
420     eval_a = evaluate_variant(videos, variant_a, iou, honest)
421     eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                       agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478               for m in mr.matches]
479     agree_iou = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

```

```

480
481 def _dur(ivs: List[Interval]) -> float:
482     return sum(e - s for s, e in ivs)
483
484     return {
485         "video": video.name,
486         "agree_iou": agree_iou,
487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                     timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                     "spec_hash": result["variant_a"]["spec_hash"],
518                     "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                     "spec_hash": result["variant_b"]["spec_hash"],
521                     "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%0.3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,

```

```

544             gts: Optional[List[Interval]] = None,
545             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558         rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563         rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566         rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568         gts=gts)
569

```

33. assistant (2026-06-18T01:34:18.913Z)

Now let me focus on examining the bootstrap_ci CI bounds math carefully against the known issue in `paired\_delta\_ci`, and look at the `merge\_overlaps`, `label\_candidates`, and `gemini\_refine` functions for boundary conditions. Let me also check `score\_calibration.py` (Platt/isotonic) which is statistical.

34. user (2026-06-18T01:34:20.336Z)

```

1         """Per-cue score calibration (course-correction C2) ? make ``confidence`` mean
2         something.
3
4         A CNN softmax, an audio onset score, and an integer net-crossing count live on wildly
5         different scales. Fed raw into the learned arbiter (the ADR-004 logistic regression),
6         the
7         most over-confident cue dominates and starves complementary cues. Calibrating each
8         producer's score to a probability *before* it becomes a feature column makes the
9         ``(value, confidence)`` semantics real and the arbiter's weights stable. See
10        ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C2.
11
12        Two calibrators, both standard:
13        - **Platt scaling** ? fit ``p = sigmoid(a*s + b)`` (parametric; good for monotone,
14        roughly-sigmoidal cues; uses Platt's target smoothing to avoid 0/1 overfitting).
15        - **Isotonic regression** (pool-adjacent-violators) ? non-parametric monotone fit; more
16        flexible, needs a bit more data.
17
18        Plus ``brier_score`` and ``expected_calibration_error`` to *measure* calibration. All
19        **additive**, pure (stdlib only), deterministic ? fits run in the numpy-free lanes too.
20        Calibrators are tiny dataclasses (their params are JSON-friendly), so a fitted
21        calibrator
22        can be pinned alongside a variant the way the classifier is.
23        """
24        from __future__ import annotations
25
26        import bisect
27        import math
28        from dataclasses import dataclass

```

```

26     from typing import List, Sequence, Tuple
27
28     _EPS = 1e-12
29
30
31     def _sigmoid(z: float) -> float:
32         if z >= 0:
33             return 1.0 / (1.0 + math.exp(-z))
34         e = math.exp(z)
35         return e / (1.0 + e)
36
37
38     def _clip01(p: float) -> float:
39         return min(1.0 - _EPS, max(_EPS, p))
40
41
42     @dataclass(frozen=True)
43     class PlattCalibrator:
44         """`p = sigmoid(a*score + b)` . Call it on a raw cue score to get a probability."""
45         a: float
46         b: float
47
48         def __call__(self, s: float) -> float:
49             return _clip01(_sigmoid(self.a * s + self.b))
50
51
52     @dataclass(frozen=True)
53     class IsotonicCalibrator:
54         """Piecewise-constant monotone map from raw score to probability (PAVA fit).
55
56         ``xs`` are sorted training scores, ``ys`` the non-decreasing fitted probabilities;
57         a query returns the fitted value of the largest ``x <= score`` (clipped at the
ends).
58         """
59         xs: List[float]
60         ys: List[float]
61
62         def __call__(self, s: float) -> float:
63             if not self.xs:
64                 return 0.5
65             idx = bisect.bisect_right(self.xs, s) - 1
66             if idx < 0:
67                 idx = 0
68             return self.ys[idx]
69
70
71     def fit_platt(scores: Sequence[float], labels: Sequence[int],
72                  iters: int = 200, lr: float = 0.5) -> PlattCalibrator:
73         """Fit Platt scaling by gradient descent on (smoothed) log-loss.
74
75         Scores are standardised internally for a stable fit, then folded back so the
returned
76         ``a``/``b`` apply to the raw score. Platt target smoothing
77         ( $t_+ = (N+1)/(N+2)$ ,  $t_- = 1/(N+2)$ ) keeps the fit off the 0/1 rails.
78         Degenerate inputs (empty, zero-variance, single class) fall back to a sane constant
map.
79         """
80         n = len(scores)
81         if n == 0 or n != len(labels):
82             return PlattCalibrator(a=0.0, b=0.0) # ? p = 0.5 everywhere
83         mean = sum(scores) / n
84         var = sum((s - mean) ** 2 for s in scores) / n
85         sd = math.sqrt(var)
86         n_pos = sum(1 for y in labels if y >= 0.5)
87         n_neg = n - n_pos

```

```

88     if sd <= _EPS or n_pos == 0 or n_neg == 0:
89         # No usable score spread or one class: best constant probability = base rate.
90         base = _clip01(n_pos / n)
91         return PlattCalibrator(a=0.0, b=math.log(base / (1.0 - base)))
92     t_pos = (n_pos + 1.0) / (n_pos + 2.0)
93     t_neg = 1.0 / (n_neg + 2.0)
94     z = [(s - mean) / sd for s in scores]
95     targets = [t_pos if y >= 0.5 else t_neg for y in labels]
96
97     a, b = 1.0, 0.0 # in standardised space
98     for _ in range(max(1, iters)):
99         ga = gb = 0.0
100         for zi, t in zip(z, targets):
101             p = _sigmoid(a * zi + b)
102             d = p - t
103             ga += d * zi
104             gb += d
105         a -= lr * ga / n
106         b -= lr * gb / n
107     # Fold standardisation back to raw score:  $z = (s - \text{mean}) / \text{sd}$  ?  $a * z + b = (a / \text{sd}) * s + (b - a * \text{mean} / \text{sd})$ .
108     return PlattCalibrator(a=a / sd, b=b - a * mean / sd)
109
110
111     def _pava(ys: Sequence[float]) -> List[float]:
112         """Pool-adjacent-violators: nearest non-decreasing fit to ``ys`` (in given
113         order)."""
114         blocks: List[List[float]] = [] # each [value, weight]
115         for y in ys:
116             blocks.append([float(y), 1.0])
117             while len(blocks) >= 2 and blocks[-2][0] > blocks[-1][0]:
118                 v2, w2 = blocks.pop()
119                 v1, w1 = blocks.pop()
120                 blocks.append([(v1 * w1 + v2 * w2) / (w1 + w2), w1 + w2])
121         fitted: List[float] = []
122         for value, weight in blocks:
123             fitted.extend([value] * int(round(weight)))
124         return fitted
125
126     def fit_isotonic(scores: Sequence[float], labels: Sequence[int]) -> IsotonicCalibrator:
127         """Fit isotonic (monotone) calibration via PAVA on score-sorted labels."""
128         n = len(scores)
129         if n == 0 or n != len(labels):
130             return IsotonicCalibrator(xs=[], ys=[])
131         order = sorted(range(n), key=lambda i: (scores[i], i))
132         xs = [float(scores[i]) for i in order]
133         ys = [_clip01(v) for v in _pava([float(labels[i]) for i in order])]
134         return IsotonicCalibrator(xs=xs, ys=ys)
135
136
137     def brier_score(probs: Sequence[float], labels: Sequence[int]) -> float:
138         """Mean squared error of probabilities vs 0/1 labels (lower = better calibrated)."""
139         n = len(probs)
140         if n == 0 or n != len(labels):
141             return 0.0
142         return sum((p - y) ** 2 for p, y in zip(probs, labels)) / n
143
144
145     def expected_calibration_error(probs: Sequence[float], labels: Sequence[int],
146                                   n_bins: int = 10) -> float:
147         """Expected calibration error: |accuracy - confidence| per equal-width bin,
148         weighted."""
149         n = len(probs)
150         if n == 0 or n != len(labels):

```

```

150         return 0.0
151     bins = max(1, n_bins)
152     bin_sum_p = [0.0] * bins
153     bin_sum_y = [0.0] * bins
154     bin_cnt = [0] * bins
155     for p, y in zip(probs, labels):
156         b = min(bins - 1, int(p * bins))
157         bin_sum_p[b] += p
158         bin_sum_y[b] += y
159         bin_cnt[b] += 1
160     ece = 0.0
161     for i in range(bins):
162         if bin_cnt[i]:
163             conf = bin_sum_p[i] / bin_cnt[i]
164             acc = bin_sum_y[i] / bin_cnt[i]
165             ece += (bin_cnt[i] / n) * abs(acc - conf)
166     return ece
167
168
169 def calibration_report(probs: Sequence[float], labels: Sequence[int],
170                       n_bins: int = 10) -> Tuple[float, float]:
171     """Convenience: ``(brier, ece)`` for a set of probabilities vs labels."""
172     return brier_score(probs, labels), expected_calibration_error(probs, labels, n_bins)
173

```

35. user (2026-06-18T01:34:20.789Z)

```

1     """Build a training set for the offline, API-free learned rally segmenter.
2
3     Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4     a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5     produces candidate rally windows with features; the collector's ground-truth
6     rally intervals tell us which windows are real. A lightweight classifier learns
7     that mapping, replacing the paid Gemini refinement with a local model.
8
9     This module is the *data layer* ? pure and unit-tested:
10
11     - `label_candidates` ? generic: label windows positive/negative by whether they
12       match a ground-truth rally (same IoU matching the evaluator uses, so training
13       labels are consistent with how we score).
14     - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15       persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16     - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18     No model training or I/O here; that lives in the trainer/segmenter once the
19     substrate is chosen from the Phase 3 candidate-recall result.
20     """
21
22     from typing import Callable, Dict, List, Tuple
23
24     from backend.eval.metrics import Interval, match_intervals
25
26     # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27     # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28     YOLO_FEATURE_NAMES = [
29         "duration", "peak_velocity", "mean_velocity", "active_frame_count",
30         "track_id_count",
31     ]
32
33     def extract_yolo_features(row: Dict) -> List[float]:
34         """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36         `stats` is already JSON-deserialized by the DB layer. Missing fields default
37         to 0.0 so a sparse/old row still yields a well-formed vector.

```

```

38     """
39     stats = row.get("stats") or {}
40     duration = float(row["end_time"]) - float(row["start_time"])
41     return [
42         duration,
43         float(stats.get("peak_velocity", 0.0)),
44         float(stats.get("mean_velocity", 0.0)),
45         float(stats.get("active_frame_count", 0)),
46         float(stats.get("track_id_count", 0)),
47     ]
48
49
50 def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                     iou_threshold: float = 0.5) -> List[int]:
52     """Label each candidate window 1 (real rally) or 0 (spurious).
53
54     A window is positive iff it is matched one-to-one to a ground-truth rally at
55     `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56     so the classifier is trained on exactly what we score. Substrate-agnostic:
57     works for YOLO or WASB-derived windows alike.
58     """
59     mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60     positive = {m.pred_index for m in mr.matches}
61     return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65     candidate_rows: List[Dict],
66     gt_intervals: List[Interval],
67     iou_threshold: float = 0.5,
68     feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69 ) -> Tuple[List[List[float]], List[int], List[Interval]]:
70     """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72     `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73     features without changing the labeling logic. Returns the feature matrix `X`,
74     binary labels `y`, and the candidate intervals (handy for downstream merging).
75     """
76     intervals: List[Interval] = [
77         (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78     ]
79     y = label_candidates(intervals, gt_intervals, iou_threshold)
80     X = [feature_fn(r) for r in candidate_rows]
81     return X, y, intervals
82

```

36. assistant (2026-06-18T01:34:23.939Z)

Let me examine `merge\_overlaps` in `gemiini_refine.py` and the `regression.py` `gt\_hash`, plus `eval\_partial\_labels` flow already reviewed. Let me also check `calibrate_wasb.py` `load\_golden\_gts`.

37. user (2026-06-18T01:34:25.315Z)

```

1     """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3     WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4     EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5     each candidate window is cut to a small padded clip and handed to Gemini, which ?
6     seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7     nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8     not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10    Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11    sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,

```

```

12     calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14     This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15     trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17     Run (GEMINI_API_KEY in env):
18     python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
19 \
20     --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
21
22     """
23     from __future__ import annotations
24
25     import os
26     import os.path as osp
27     import time
28     import argparse
29     import concurrent.futures
30     from typing import List, Tuple, Optional
31
32     from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
33     from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
34     from backend.utils.video import get_video_duration, extract_video_chunk
35     from backend.sports import sport_registry
36     from backend.providers import provider_registry
37
38     Interval = Tuple[float, float]
39
40     def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
41         """Union of overlapping intervals, optionally stitching across a small gap.
42
43         gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
44         shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
45         gaps
46         (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
47         logic, which over-merged multiple real rallies WASB had bundled into one candidate.)"""
48         merged: List[Interval] = []
49         for s, e in sorted(intervals):
50             if merged and s <= merged[-1][1] + gap_tol:
51                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
52             else:
53                 merged.append((s, e))
54         return merged
55
56     def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
57         """Map a Gemini clip's returned segments back to full-video seconds, clamped to
58         clip."""
59         out: List[Interval] = []
60         for sg in raw or []:
61             try:
62                 st = float(sg["start_time"]) + clip_start
63                 en = float(sg["end_time"]) + clip_start
64             except (TypeError, ValueError, KeyError):
65                 continue
66             st = max(clip_start, st)
67             en = min(clip_end, en)
68             if en > st:
69                 out.append((round(st, 3), round(en, 3)))
70         return out
71
72     def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,

```



```

72         pad: float, duration: float, tmpdir: str, idx: int,
73         clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
List[Interval]]:
74     # Own provider/client per call: the genai client isn't reliably thread-safe, and a
75     # shared one across workers throws socket errors. A fresh lazy client is cheap.
76     provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
77     s, e = window
78     cs = max(0.0, s - pad)
79     ce = min(duration, e + pad) if duration > 0 else e + pad
80     clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
81     # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
82     # 500MB upload cap even for 4K/5K sources.
83     if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
scale_width=clip_scale_width):
84         return idx, window, []
85     try:
86         prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
87         raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
88         mapped = _offset_segments(raw, cs, ce)
89     finally:
90         try:
91             os.remove(clip)
92         except OSError:
93             pass
94     # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
95     # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
96     # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
97     # by the gap-tolerant global merge in run().
98     return idx, window, mapped
99
100
101 def run(video: str, trajectory: str, fps: float, frame_width: float,
102         cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
103         max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
104         min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
dict:
105     api_key = os.environ.get("GEMINI_API_KEY")
106     if not api_key:
107         raise SystemExit("GEMINI_API_KEY not set in env")
108     sport_profile = sport_registry.get("badminton")
109     duration = get_video_duration(video)
110
111     candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
112     if limit:
113         candidates = candidates[:limit]
114     tmpdir = osp.join("output", "gemini_refine_clips")
115     os.makedirs(tmpdir, exist_ok=True)
116
117     print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
{model}, "
118         f"pad {pad}s, {max_workers} workers", flush=True)
119     t0 = time.time()
120     per_candidate = []
121     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
122         futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
123                         tmpdir, i, clip_scale_width)
124                 for i, w in enumerate(candidates)]
125     for fut in concurrent.futures.as_completed(futs):
126         idx, window, mapped = fut.result()
127         per_candidate.append((idx, window, mapped))
128         tag = "REJECT (no rally)" if not mapped else \

```

```

129         " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
130         print(f"  cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)
131
132         raw_intervals = [iv for _, _, m in per_candidate for iv in m]
133         refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
134         write_segments_csv(out_csv, refined)
135         try:
136             os.rmdir(tmpdir)
137         except OSError:
138             pass
139
140         kept_cands = sum(1 for _, _, m in per_candidate if m)
141         play = sum(e - s for s, e in refined)
142         print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
143             f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
144             f"({time.time() - t0:.0f}s)", flush=True)
145         return {"refined": refined, "num_candidates": len(candidates),
146             "candidates_with_rally": kept_cands, "out_csv": out_csv}
147
148
149     def full_video_rallies(video: str, model: str = "gemini-flash-latest",
150         out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
151         min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
152         max_workers: int = 3) -> List[Interval]:
153         """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
154         upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
155         Independent of the trajectory; complements refine()."""
156         api_key = os.environ.get("GEMINI_API_KEY")
157         if not api_key:
158             raise SystemExit("GEMINI_API_KEY not set in env")
159         sport_profile = sport_registry.get("badminton")
160         duration = get_video_duration(video)
161         starts, t = [], 0.0
162         while t < duration:
163             starts.append(t)
164             t += max(1.0, chunk_sec - overlap)
165         tmpdir = osp.join("output", "gemini_full_clips")
166         os.makedirs(tmpdir, exist_ok=True)
167
168         def do_chunk(ci: int, cstart: float) -> List[Interval]:
169             cend = min(cstart + chunk_sec, duration)
170             clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
171             if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
172                 return []
173             try:
174                 provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
175                 prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
176                 raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
177                 return _offset_segments(raw, cstart, cend)
178             finally:
179                 try:
180                     os.remove(clip)
181                 except OSError:
182                     pass
183

```

```

184         print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
185         f"-> Gemini {model}", flush=True)
186         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
187             chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
188             rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
189             write_segments_csv(out_csv, rallies)
190             try:
191                 os.rmdir(tmpdir)
192             except OSError:
193                 pass
194             play = sum(e - s for s, e in rallies)
195             print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",
flush=True)
196             return rallies
197
198
199     def main() -> None:
200         ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
201         ap.add_argument("--video", required=True)
202         ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
203         ap.add_argument("--fps", type=float, default=59.94)
204         ap.add_argument("--frame-width", type=float, default=1920.0)
205         ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
206             help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
207         ap.add_argument("--pad", type=float, default=3.0)
208         ap.add_argument("--model", default="gemini-flash-latest")
209         ap.add_argument("--max-workers", type=int, default=4)
210         ap.add_argument("--min-dur", type=float, default=2.0)
211         ap.add_argument("--out", default="output/gemini_refined.csv")
212         ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
213         ap.add_argument("--clip-scale-width", type=int, default=1280,
214             help="downscale uploaded clips to this width (0=keep source res)")
215         ap.add_argument("--full-video", action="store_true",
216             help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini "
217             "instead of refining WASB candidates (needs no --trajectory)")
218         args = ap.parse_args()
219         scale = args.clip_scale_width or None
220         if args.full_video:
221             full_video_rallies(args.video, model=args.model, out_csv=args.out,
222                 clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
223             return
224         if not args.trajectory:
225             raise SystemExit("--trajectory is required unless --full-video")
226         cfg = BASELINE if args.config == "baseline" else TUNED
227         run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228             model=args.model, max_workers=args.max_workers, out_csv=args.out,
229             min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
230
231
232     if __name__ == "__main__":
233         main()
234

```

38. user (2026-06-18T01:34:25.825Z)

```

1         """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".

```

```

2
3 Wires the already-tested pieces together:
4 WASB trajectory CSV (Frame,Visibility,X,Y)
5     -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6     -> backend.eval.calibration (improvement_report / cross_validate)
7
8 Run (after a full-video WASB pass exists):
9     python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12 Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13 same GT is an **optimistic** upper bound. The trustworthy number is the
14 cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15 `leave_one_video_out` for precision/F1 generalization).
16 """
17 from __future__ import annotations
18
19 import argparse
20 from typing import List, Tuple, Callable
21
22 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23 from backend.eval.calibration import (
24     select_best, improvement_report, cross_validate, evaluate,
25 )
26
27 Interval = Tuple[float, float]
28 Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
29
30 BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33 def load_golden_gts(csv_path: str) -> List[Interval]:
34     """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36     Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37     start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39     """
40     from backend.eval.gt_loader import load_gt_intervals
41     return load_gt_intervals(csv_path, strict=False)
42
43
44 def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48     def predict_fn(cfg: Config) -> List[Interval]:
49         vt, mg, mwd = cfg
50         wins = TrackNetRunner.trajectory_to_action_windows(
51             points, fps=fps, frame_width=frame_width,
52             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53         )
54         return [(w["start"], w["end"]) for w in wins]
55
56     return predict_fn
57
58
59 def default_grid() -> List[Config]:
60     return [(vt, mg, mwd)
61             for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62             for mg in (1.0, 2.0, 3.0)
63             for mwd in (1.0, 2.0, 3.0)]

```

```

64
65
66     def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67             iou: float = 0.5) -> dict:
68         gts = load_golden_gts(labels_csv)
69         if not gts:
70             raise SystemExit(f"no usable golden rallies in {labels_csv}")
71         predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
72         grid = default_grid()
73
74         base = evaluate(predict_fn(BASELINE), gts, iou)
75         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
77         cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
out recall
78
79         print("=== WASB rally-segmentation calibration ===")
80         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
81         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
82               f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83         print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
84         print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
85               f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86         print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
87               f"({fit['f1_delta_pct']:+.1f}%)")
88         print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89               f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90               f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91         print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
92         return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
93
94
95     def main():
96         ap = argparse.ArgumentParser()
97         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99         ap.add_argument("--fps", type=float, default=59.94)
100        ap.add_argument("--frame-width", type=float, default=1920.0)
101        ap.add_argument("--iou", type=float, default=0.5)
102        args = ap.parse_args()
103        run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106    if __name__ == "__main__":
107        main()
108

```

39. assistant (2026-06-18T01:34:39.446Z)

I've now reviewed the core numeric/statistical paths. Let me re-examine a couple of suspicious spots more carefully before concluding.

First, the `\_offset\_segments` in `gemini_refine` (lines 55-68): it computes `en = float(sg["end_time"]) + clip_start`` ? note it adds `clip_start`` to `end_time`, not `clip_end``. That's correct since Gemini returns relative-to-clip times and the clip starts at `clip_start``. Fine.

Now let me scrutinize the R4 `_trim_inactive_tail`` sliding window more carefully ? this is the

kind of windowing logic where an off-by-one matters. And let me re-examine the ``over_seg_guard`` ``aligned`` function: when names aren't present in all rows, it falls back to positional alignment, but ``compare`` always passes rows that have ``name``. Let me verify the ``merge_split_report`` connected-components with ``complex`` classification edge: ``gts_split`` only counts ``+1`` per split component regardless of how many GTs ? but split is ``(>1, 1)`` so exactly 1 GT, fine. ``split_rate`` = `gts_split / m``. OK.

Let me look at the ``_split_overlong_groups`` `hard_cap` branch more carefully for the midpoint split. When ``hard_cap`` and a group can't be split by gap, it computes ``mid`` and ``split_i`` = `min(range(1, len(g)), key=lambda i: abs(g[i][0]-mid))``. If ``split_i`` could be ``len(g)`` (the last index), then ``g[split_i:]`` would be empty and ``g[:split_i]`` would be the whole group ? infinite loop. Can ``split_i == len(g)``? ``range(1, len(g))`` goes up to ``len(g)-1``, so ``split_i <= len(g)-1``. So ``g[split_i:]`` has at least 1 element and ``g[:split_i]`` has at least 1. Both non-empty. But could one piece still be the full length causing non-termination? If ``split_i=1``, then ``g[:1]`` (length 1) and ``g[1:]`` (length `len-1``). The length-1 piece terminates; the other is strictly smaller. Progress guaranteed. Good.

Now let me check the `blob_fallback` similar logic ? same structure, ``split_i`` in ``range(1, len(h))``, fine.

Let me look at the ``bootstrap_ci`` more carefully for a real edge case, and check the `regression.py` and `batch.py` files.

40. user (2026-06-18T01:34:40.922Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7          ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8          predictions  ? the production pipeline's stored DB rows   [backend.eval.harness]
9          comparison   ? interval matching + P/R/F1                 [backend.eval.metrics]
10         baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
39         different labels."""
40         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
```

```

40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
detected
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100                    _baseline_key(video_id, stage), precision, recall)
101

```

```

102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105            stage: str = "final", iou_threshold: float = 0.5,
106            detector: Optional[str] = None,
107            tolerance: float = DEFAULT_TOLERANCE,
108            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148     else:
149         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )

```



```

158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                           tier: str = "file", stage: str = "final",
162                           iou_threshold: float = 0.5,
163                           detector: Optional[str] = None,
164                           tolerance: float = DEFAULT_TOLERANCE,
165                           baseline_path: str = DEFAULT_BASELINE_PATH,
166                           guideline: Optional[str] = None,
167                           sport: str = "badminton",
168                           provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172     once those providers land). `video_ref` is whatever that provider expects (a CSV
173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, *(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                   detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182 def format_result(r: RegressionResult) -> str:
183     verdict = "REGRESSED" if r.regressed else "ok"
184     head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185            f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186     if r.baseline_precision is not None:
187         head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188     return head + ".join(f'\n    - {why}' for why in r.reasons)
189

```

41. user (2026-06-18T01:34:41.671Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7     across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8     only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9     merges overlaps, and avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10    LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11    `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12    path simply never calls into it.)
13
14    The replay reuses the already-shipped eval stack VERBATIM::
15
16        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17        ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20    Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22    * Fixtures are synthetic ? deterministic formulae, NO randomness ? so they carry
zero

```

```

23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true`,
    ``license="synthetic"`.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
    ``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     $6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45     parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     ""
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
57     import shutil
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
    -----
65     from backend.eval import distill_local
66     from backend.eval.experiment import CONTROL, VideoCandidates, predict
67     from backend.eval.gt_loader import load_gt_intervals
68     from backend.eval.metrics import score
69     from backend.eval.regression import gt_hash

```

```

70     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
71
72     Interval = Tuple[float, float]
73
74     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
75     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
76     CURRENT_SCHEMA = 1
77
78     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
79     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
80     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
81
82     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
83     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
84     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
85     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
86     #: re-states these in its provenance; the replay reads them from there.
87     PINNED_VT = 0.02
88     PINNED_MERGE_GAP = 2.0
89     PINNED_MIN_WINDOW = 2.0
90     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
91     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
92     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
93     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
94     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
95     PINNED_MAX_WINDOW = 0.0
96
97     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
98     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
99     SHUTTLE_FEATURES: Tuple[str, ...] = (
100         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101     )
102     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104     )
105
106     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
108     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
109
110     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
111     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
112
113     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
114     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
115     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
116
117     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,

```

```

118     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
119     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
120     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
121     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
122
123
124     # =====
125     # Synthetic trajectory generation (deterministic ? NO random)
126     # =====
127
128
129     @dataclass(frozen=True)
130     class RallySpec:
131         """One in-flight rally segment: the shuttle oscillates across the net midline.
132
133         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
134         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
135         net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
136         """
137
138         start_frame: int
139         n_frames: int
140         period: float = 30.0
141         amplitude: float = 400.0
142
143
144     @dataclass(frozen=True)
145     class GapSpec:
146         """Dead-time between rallies ? what separates one window from the next.
147
148         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
149         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
150         end an active run so the next rally forms a distinct window.
151         """
152
153         start_frame: int
154         n_frames: int
155         kind: str = "still" # "still" | "occluded"
156
157
158     @dataclass(frozen=True)
159     class FixtureSpec:
160         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
161
162         ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
163         "labels.csv". For a synthetic fixture they line up with the rallies by construction,
164         which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
165         field-quality one (§6).
166         """
167
168         slug: str
169         description: str
170         fps: float
171         frame_width: float
172         frame_height: float
173         segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
174         gts: Tuple[Interval, ...]
175         net_midline: float = 640.0
176
177
178     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float

```

```

179         ) -> List[Tuple[int, int, float, float]]:
180         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
181         rows: List[Tuple[int, int, float, float]] = []
182         y_mid = frame_height / 2.0
183         for i in range(spec.n_frames):
184             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
185             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
186             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
187             rows.append((spec.start_frame + i, 1, x, y))
188         return rows
189
190
191     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
192         ) -> List[Tuple[int, int, float, float]]:
193         """Dead-time rows: resting-visible (still) or absent (occluded)."""
194         rows: List[Tuple[int, int, float, float]] = []
195         y_mid = frame_height / 2.0
196         for i in range(spec.n_frames):
197             if spec.kind == "occluded":
198                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
199             elif spec.kind == "still":
200                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
201             else: # pragma: no cover - guarded by builder
202                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
203         return rows
204
205
206     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
207         """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
208         rows: List[Tuple[int, int, float, float]] = []
209         for seg in spec.segments:
210             if isinstance(seg, RallySpec):
211                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
212             elif isinstance(seg, GapSpec):
213                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
214             else: # pragma: no cover - guarded by dataclass shape
215                 raise TypeError(f"unknown segment type {type(seg).__name__}")
216         rows.sort(key=lambda r: r[0])
217         return rows
218
219
220     def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
221         """Write trajectory rows as the TrackNet CSV ``(Frame,Visibility,X,Y)``, LF + 6dp
floats."""
222         lines = ["Frame,Visibility,X,Y"]
223         for frame, vis, x, y in rows:
224             lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
225         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
226
227
228     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
230         lines = ["start,end"]
231         for s, e in gts:
232             lines.append(f"{_fmt(s)},{_fmt(e)}")
233         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236     # =====
237     # Built-in synthetic scenarios (the 3 committed slugs)

```

```

238 # =====
239
240
241 def _fr(seconds: float, fps: float) -> int:
242     return int(round(seconds * fps))
243
244
245 def builtin_specs() -> List[FixtureSpec]:
246     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247     windowing behaviour (one window / dead-time split / occlusion split)."""
248     fps = 30.0
249     fw, fh = 1280.0, 720.0
250     net = 640.0
251
252     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253     single = FixtureSpec(
254         slug="clean_single_rally",
255         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257         segments=(RallySpec(start_frame=0, n_frames=120),),
258         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259         gts=((0.0, 4.0),),
260     )
261
262     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264     r0 = RallySpec(start_frame=0, n_frames=120)
265     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267     multi = FixtureSpec(
268         slug="multi_rally_with_deadtime",
269         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271         segments=(r0, gap_still, r1),
272         gts=((0.0, 4.0), (7.0, 11.0)),
273     )
274
275     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276     # visibility break ? proves occlusion (not just low velocity) separates windows.
277     o0 = RallySpec(start_frame=0, n_frames=120)
278     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280     occ = FixtureSpec(
281         slug="occlusion_break",
282         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
283         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
284         segments=(o0, gap_occ, o1),
285         gts=((0.0, 4.0), (7.0, 11.0)),
286     )
287
288     return [single, multi, occ]
289
290
291 def _spec_by_slug() -> Dict[str, FixtureSpec]:
292     return {s.slug: s for s in builtin_specs()}
293
294
295 # =====

```

```

296 # The shared replay (CONTROL / pure-stdlib path)
297 # =====
298
299
300 def _fmt(x: float) -> str:
301     """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
302     ``0.0``."""
303     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
304     if r == 0.0:
305         r = 0.0
306     return f"{r:.6f}"
307
308 def _round(x: float) -> float:
309     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
310     return 0.0 if r == 0.0 else r
311
312
313 def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
314     """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
315     when present
316     (the pinned params travel with the fixture), else the module-level pinned defaults.
317     NEVER
318     ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
319     read here ?
320     ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
321     path; the
322     pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
323     PINNED_MAX_WINDOW)."""
324     if prov and "windowing" in prov:
325         w = prov["windowing"]
326         return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
327     return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
328
329
330 def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
331     """Replay one fixture through the shipped CONTROL path; return its recomputed
332     snapshot.
333     Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
334     committed
335     ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
336     parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
337     shuttle
338     cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
339
340     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
341     control_segments:[{s,e}], score:{...}, gt_hash}``. CONTROL keeps every candidate
342     then
343     merges overlaps; NO numpy is touched.
344     """
345     fdir = _resolve_dir(slug_or_dir)
346     slug = fdir.name
347     traj_csv = fdir / "trajectory.csv"
348     labels_csv = fdir / "labels.csv"
349     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
350 else None
351
352     fps, fw = _fps_fw(prov, slug)
353     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
354     proposal = _pinned_proposal(prov)
355
356     intervals, feature_rows = distill_local.build_candidates(
357         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)

```

```

350
351     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354     features: Dict[str, List[float]] = {
355         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356     }
357     gts = load_gt_intervals(str(labels_csv), strict=True)
358     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361     sc = score(preds, gts, iou)
362
363     candidates: List[Dict[str, Any]] = []
364     for j, (s, e) in enumerate(intervals):
365         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370     return {
371         "slug": slug,
372         "schema_version": CURRENT_SCHEMA,
373         "iou": _round(iou),
374         "candidates": candidates,
375         "control_segments": [[_round(s), _round(e)] for s, e in preds],
376         "score": {
377             "iou_threshold": _round(sc.iou_threshold),
378             "num_pred": int(sc.num_pred),
379             "num_gt": int(sc.num_gt),
380             "true_positives": int(sc.true_positives),
381             "false_positives": int(sc.false_positives),
382             "false_negatives": int(sc.false_negatives),
383             "precision": _round(sc.precision),
384             "recall": _round(sc.recall),
385             "f1": _round(sc.f1),
386             "mean_iou": _round(sc.mean_iou),
387             "mean_start_error": _round(sc.mean_start_error),
388             "mean_end_error": _round(sc.mean_end_error),
389         },
390         "gt_hash": gt_hash(gts),
391     }
392
393
394     # =====
395     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396     # =====
397
398
399     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400         ) -> Dict[str, Any]:
401         """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, $5)."""
402         desc = ""
403         if isinstance(spec_or_dir, FixtureSpec):
404             desc = spec_or_dir.description
405         return {
406             "slug": slug,

```



```

407         "schema_version": CURRENT_SCHEMA,
408         "kind": "synthetic",
409         "license": "synthetic",
410         "minors_free": True,
411         "description": desc,
412         "fps": _round(fps),
413         "frame_width": _round(fw),
414         "iou": _round(iou),
415         "windowing": {
416             "vt": PINNED_VT,
417             "mg": PINNED_MERGE_GAP,
418             "mwd": PINNED_MIN_WINDOW,
419             "max_win": PINNED_MAX_WINDOW,
420         },
421         "paths": {
422             "trajectory": "trajectory.csv",
423             "labels": "labels.csv",
424             "expected": "expected.json",
425         },
426         "note": (
427             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
428             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
429         ),
430     }
431
432
433     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
434                       iou: float = 0.5) -> Dict[str, Any]:
435         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
436         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
437
438         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
439         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
440         """
441         fdir = fixtures_dir / slug
442         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
443         if not traj_csv.is_file() or not labels_csv.is_file():
444             raise FileNotFoundError(
445                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
446
447         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
448         spec = _spec_by_slug().get(slug)
449         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
450         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
451         _write_json(fdir / "provenance.json", prov)
452
453         expected = replay_fixture(fdir)
454         _write_json(fdir / "expected.json", expected)
455         return expected
456
457
458     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
459         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
460         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461         specs = builtin_specs()
462         fixtures_dir.mkdir(parents=True, exist_ok=True)
463         slugs: List[str] = []
464         for spec in specs:

```

```

465         fdir = fixtures_dir / spec.slug
466         fdir.mkdir(parents=True, exist_ok=True)
467         rows = make_synthetic_trajectory(spec)
468         write_trajectory_csv(rows, fdir / "trajectory.csv")
469         write_labels_csv(spec.gts, fdir / "labels.csv")
470         freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471         slugs.append(spec.slug)
472     _write_manifest(specs, fixtures_dir)
473     return slugs
474
475
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
479         labels.csv ?
480         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
481         regression; the
482         mitigation is reviewing the visible diff + recording a reason (the CLI requires
483         one)."""
484         if all_slugs:
485             targets = [d.name for d in sorted(fixtures_dir.iterdir())
486                       if d.is_dir() and (d / "trajectory.csv").is_file()]
487         elif slug:
488             targets = [slug]
489         else:
490             raise ValueError("refresh_snapshot needs --slug S or --all")
491         for s in targets:
492             freeze_fixture(s, fixtures_dir=fixtures_dir)
493         return targets
494
495     # =====
496     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
497     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
498     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
499     # =====
500
501     class RefusalError(ValueError):
502         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
503
504         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
505         it) that
506         names *why* the unsafe public-fixture path was refused. The refusal is the code-
507         enforced
508         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
509         minors-excluded
510         / license flags are human-asserted, but the gate makes the unsafe path impossible to
511         reach
512         *silently* ? a fixture cannot be written unless all three are affirmatively set.
513         """
514
515     def _new_surrogate_slug() -> str:
516         """An opaque RANDOM surrogate slug ? ``fx<16 hex>`` from ``os.urandom`` (§4c).
517
518         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
519         random,
520         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
521         over a
522         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
523         off-git.
524         """
525         return f"fx_{os.urandom(8).hex()}"

```

```

520
521     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         """
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
552         for sp in source_paths:
553             if not sp:
554                 continue
555             p = Path(sp)
556             for token in (sp, p.name, p.stem):
557                 t = (token or "").strip().lower()
558                 # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559                 # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560                 # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561                 # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562                 # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563                 if not t or t in _SCHEMA_FILENAME_LITERALS:
564                     continue
565                 if t in low:
566                     raise RefusalError(
567                         f"{where}: source-path token {token!r} found in written output ? the
source "

```

```

568         f"filename/path must never be persisted ($4c metadata strip).")
569
570
571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                                ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
574         ``(Frame,Visibility,X,Y)`` rows.
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
576         invariant under a
577         pure time shift); only the absolute frame index moves toward 0, severing the match
578         timeline.
579         """
580         rows: List[Tuple[int, int, float, float]] = []
581         for p in points:
582             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
583             float(p.y)))
584         rows.sort(key=lambda r: r[0])
585         return rows
586
587     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
588         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
589         invariant).
590         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
591         Each label
592         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
593         of two exact
594         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
595         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
596         reset score
597         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
598         ``freeze_real_fixture``
599         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
600         second
601         offset (the grid, not a real metric divergence)."""
602         return [(round(s) - t0_sec, round(e) - t0_sec) for s, e in gts]
603
604
605     def freeze_real_fixture(
606         label: str,
607         trajectory_csv_path: Any,
608         labels_csv_path: Any,
609         *,
610         license: str,
611         minors_free: bool,
612         owned: bool,
613         fps: float,
614         frame_width: float,
615         fixtures_dir: Path = FIXTURES_DIR,
616         slug: Optional[str] = None,
617         time_origin_reset: bool = True,
618         source_note: str = "",
619     ) -> Dict[str, Any]:
620         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
621         de-attributed
622         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
623         row P1.)
624
625         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
626         is emitted
627         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
628         blank string,
629         AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text

```

```

identifier like a
619     player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
620     match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621     shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622     ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623     ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625     ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626     back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627     contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628     committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629     retained for CLI compatibility / future logging.
630
631     NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632     only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633     attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634     owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636     The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637     after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638     (red-team #3).
639
640     Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641     """
642     # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
643     reasons: List[str] = []
644     if minors_free is not True:
645         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646             "of a minor is prohibited ? $3, guardrail 'exclude minors')")
647     if owned is not True:
648         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649             "broadcast/scraped = Fork B, do NOT commit ? $3 D1)")
650     if not (isinstance(license, str) and license.strip()):
651         reasons.append("license must be a non-blank string (per-record provenance tag ?
$4c "
652             "'provenance hard-fail gate')")
653     # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654     # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655     if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656         reasons.append(
657             f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
658             f"alnum + '_'/'-' , no spaces/punctuation) ? a free-text identifier
(player/event "
659             f"name) is an attribution leak; got {label!r} ($4c metadata strip, red-team
#1)")

```

```

660         if reasons:
661             raise RefusalError(
662                 "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
663                 "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
664                 + "\n - ".join(reasons))
665
666         traj_path = Path(str(trajecory_csv_path))
667         labels_path = Path(str(labels_csv_path))
668
669         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670         out_slug = slug if slug is not None else _new_surrogate_slug()
671
672         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
673         points = TrackNetRunner.parse_trajecory_csv(str(traj_path))
674         if not points:
675             raise ValueError(f"{traj_path}: no parseable trajecory rows
(Frame,Visibility,X,Y)")
676         gts = load_gt_intervals(str(labels_path), strict=True)
677
678         # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, $4c) --
679         # Subtract the SAME offset from BOTH the trajecory frames and the label seconds:
metrics.score
680         # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681         # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682         # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683         # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684         # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685         #
686         # HONEST LIMIT (not a bug): the trajecory shifts by an INTEGER frame count while
labels shift
687         # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
688         # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689         # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690         # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691         if time_origin_reset:
692             t0_frame = min(int(p.frame) for p in points)
693             t0_sec = _round(t0_frame / float(fps))
694         else:
695             t0_frame, t0_sec = 0, 0.0
696         rows = _reset_trajecory_rows(points, t0_frame)
697         reset_gts = _reset_labels(gts, t0_sec)
698
699         # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
700         # A reset label that starts < 0 means the label timeline predates the earliest
trajecory
701         # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702         # start). Refuse loudly up-front so the partial-write path is never even reached.
703         neg = [(s, e) for (s, e) in reset_gts if s < 0]
704         if neg:

```

```

705         raise RefusalError(
706             f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
707             f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708             f"earliest label must not precede the earliest trajectory frame ($4c time-
origin reset).")
709
710     iou = 0.5
711     # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712     # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713     # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714     prov = {
715         "slug": out_slug,
716         "schema_version": CURRENT_SCHEMA,
717         "kind": "cleared_real",
718         "label": str(label),
719         "license": license,
720         "minors_free": True,
721         "owned": True,
722         "fps": _round(fps),
723         "frame_width": _round(frame_width),
724         "iou": _round(iou),
725         "time_origin_reset": bool(time_origin_reset),
726         "windowing": {
727             "vt": PINNED_VT,
728             "mg": PINNED_MERGE_GAP,
729             "mwd": PINNED_MIN_WINDOW,
730             "max_win": PINNED_MAX_WINDOW,
731         },
732         "paths": {
733             "trajectory": "trajectory.csv",
734             "labels": "labels.csv",
735             "expected": "expected.json",
736         },
737         "gt_hash": gt_hash(reset_gts),
738         "note": (
739             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
740             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742             "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2).",
743         ),
744     }
745
746     # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747     # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748     # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749     # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750     fdir = fixtures_dir / out_slug
751     fixtures_dir.mkdir(parents=True, exist_ok=True)
752     if fdir.exists():
753         raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754     tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",

```

```

dir=str(fixtures_dir))
755     tmp_dir = tmp_parent / out_slug
756     tmp_dir.mkdir(parents=True, exist_ok=True)
757     try:
758         write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759         write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760         _write_json(tmp_dir / "provenance.json", prov)
761
762         expected = replay_fixture(tmp_dir)    # REUSE M1 ? correct-by-construction
snapshot
763         _write_json(tmp_dir / "expected.json", expected)
764
765         # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766         source_paths = [str(traj_path), str(labels_path)]
767         _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                         where=f"{out_slug}/expected.json", source_paths=source_paths)
769         _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                         where=f"{out_slug}/provenance.json", source_paths=source_paths)
771         # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772         if source_note:
773             _scan_forbidden(str(source_note),
774                             where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776         # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----
777         os.replace(str(tmp_dir), str(fdir))
778     finally:
779         # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780         shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782         # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783         _append_manifest_entry(prov, fixtures_dir)
784         return expected
785
786
787 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788     """A manifest row for a single fixture, derived from its provenance block."""
789     slug = prov["slug"]
790     return {
791         "slug": slug,
792         "fps": _round(float(prov["fps"])),
793         "frame_width": _round(float(prov["frame_width"])),
794         "windowing": dict(prov.get("windowing", {})),
795         "kind": prov.get("kind", "synthetic"),
796         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797         "paths": {
798             "dir": slug,
799             "trajectory": f"{slug}/trajectory.csv",
800             "labels": f"{slug}/labels.csv",
801             "expected": f"{slug}/expected.json",
802             "provenance": f"{slug}/provenance.json",
803         },
804     }
805
806
807 def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808     """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
809     entries = load_manifest(fixtures_dir)
810     entries = [e for e in entries if e.get("slug") != prov["slug"]]

```



```

811     entries.append(_manifest_entry_for(prov))
812     entries.sort(key=lambda e: str(e.get("slug")))
813     _write_json(fixtures_dir / "manifest.json", entries)
814
815
816 # =====
817 # Manifest + loaders
818 # =====
819
820
821 def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822     entries = [{
823         "slug": spec.slug,
824         "fps": _round(spec.fps),
825         "frame_width": _round(spec.frame_width),
826         "windowing": {
827             "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828             "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829         },
830         "kind": "synthetic",
831         "schema_version": CURRENT_SCHEMA,
832         "paths": {
833             "dir": spec.slug,
834             "trajectory": f"{spec.slug}/trajectory.csv",
835             "labels": f"{spec.slug}/labels.csv",
836             "expected": f"{spec.slug}/expected.json",
837             "provenance": f"{spec.slug}/provenance.json",
838         },
839     } for spec in specs]
840     _write_json(fixtures_dir / "manifest.json", entries)
841
842
843 def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844     """Load the fixtures manifest (returns [] if absent)."""
845     path = fixtures_dir / "manifest.json"
846     if not path.is_file():
847         return []
848     data = _read_json(path)
849     if not isinstance(data, list): # pragma: no cover - corrupt manifest
850         raise ValueError(f"{path}: manifest must be a JSON array")
851     return data
852
853
854 def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855     """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857     ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858     bump in one is never masked by an in-range value in the other (red-team #7)."""
859     fdir = fixtures_dir / slug
860     prov = _read_json(fdir / "provenance.json")
861     expected = _read_json(fdir / "expected.json")
862     for which, blob in (("expected", expected), ("provenance", prov)):
863         sv = int(blob.get("schema_version", 0))
864         if sv > CURRENT_SCHEMA:
865             raise ValueError(
866                 f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
867                 f"upgrade the harness")
868     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869
870
871 # =====

```

```

872     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873     # =====
874
875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
903
904         exact("slug", recomputed.get("slug"), committed.get("slug"))
905         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
906         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907         approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909         rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910         if len(rc) != len(cc):
911             out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912         else:
913             for i, (r, c) in enumerate(zip(rc, cc)):
914                 approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915                 approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916                 rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917                 if set(rs) != set(cs):
918                     out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919                 else:
920                     for k in rs:
921                         if k == "net_crossings":
922                             exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923                         else:
924                             approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926         rseg, cseg = recomputed.get("control_segments", []),

```

```

committed.get("control_segments", [])
927     if len(rseg) != len(cseg):
928         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
929     else:
930         for i, (r, c) in enumerate(zip(rseg, cseg)):
931             approx(f"control_segments[{i}][0]", r[0], c[0])
932             approx(f"control_segments[{i}][1]", r[1], c[1])
933
934     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
935     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
936     if set(rsc) != set(csc):
937         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
938     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
939     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
940                    "mean_start_error", "mean_end_error")
941     for k in int_fields:
942         exact(f"score.{k}", rsc.get(k), csc.get(k))
943     for k in float_fields:
944         approx(f"score.{k}", rsc.get(k), csc.get(k))
945
946     return out
947
948
949 # =====
950 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
951 # =====
952
953
954 def _write_json(path: Path, obj: Any) -> None:
955     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
956     path.write_text(text + "\n", encoding="utf-8", newline="\n")
957
958
959 def _read_json(path: Path) -> Any:
960     with open(path, encoding="utf-8") as f:
961         return json.load(f)
962
963
964 def _resolve_dir(slug_or_dir: Any) -> Path:
965     if isinstance(slug_or_dir, Path):
966         return slug_or_dir
967     p = Path(str(slug_or_dir))
968     if p.is_dir():
969         return p
970     return FIXTURES_DIR / str(slug_or_dir)
971
972
973 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
974     if prov and "fps" in prov and "frame_width" in prov:
975         return float(prov["fps"]), float(prov["frame_width"])
976     spec = _spec_by_slug().get(slug)
977     if spec is not None:
978         return spec.fps, spec.frame_width
979     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
980
981
982 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
983     prov_path = fdir / "provenance.json"
984     prov = _read_json(prov_path) if prov_path.is_file() else None
985     return _fps_fw(prov, slug)
986
987

```

```

988 # =====
989 # CLI
990 # =====
991
992
993 def _cmd_check(fixtures_dir: Path) -> int:
994     manifest = load_manifest(fixtures_dir)
995     if not manifest:
996         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
997               file=sys.stderr)
998         return 1
999     failures = 0
1000     for entry in manifest:
1001         slug = entry["slug"]
1002         committed = _read_json(fixtures_dir / slug / "expected.json")
1003         recomputed = replay_fixture(fixtures_dir / slug)
1004         mismatches = compare_expected(recomputed, committed)
1005         if mismatches:
1006             failures += 1
1007             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
1008                   f"(characterization = behaviour-locking, NOT correctness ? $6):")
1009             for m in mismatches:
1010                 print(f"    - {m}")
1011         else:
1012             sc = recomputed["score"]
1013             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
1014                   f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
1015     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match "
1016           f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
1017     return 0 if failures == 0 else 1
1018
1019 def main(argv: Optional[Sequence[str]] = None) -> int:
1020     ap = argparse.ArgumentParser(
1021         description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
1022 harness.")
1023     g = ap.add_mutually_exclusive_group(required=True)
1024     g.add_argument("--freeze-synthetic", action="store_true",
1025                   help="generate synthetic trajectory+labels for all built-in
1026 scenarios, then "
1027                   "freeze expected.json+provenance.json + write manifest.json")
1028     g.add_argument("--refresh-snapshot", action="store_true",
1029                   help="re-freeze expected.json from COMMITTED trajectory+labels (no
1030 regeneration)")
1031     g.add_argument("--check", action="store_true",
1032                   help="replay all fixtures + parse-compare vs committed expected.json
1033 (exit 0/1)")
1034     g.add_argument("--freeze-real", action="store_true",
1035                   help="(P1) de-attribute an OWNED real trajectory+labels into a
1036 committable Tier-0 "
1037                   "fixture; REFUSED without --owned --minors-free and a non-blank
1038 --license")
1039     ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
1040 slug for "
1041                   "--freeze-real (default: a random fx_<hex> id)")
1042     ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
1043 snapshot")
1044     ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
1045 (review the diff!)")
1046     # --freeze-real (P1 / M2 refusal gate) options:
1047     ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV
1048 (Frame,Visibility,X,Y)")
1049     ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV
1050 (start,end)")

```

```

1040     ap.add_argument("--license", dest="license_", default=None,
1041                      help="--freeze-real) the source license tag (required, non-blank)")
1042     ap.add_argument("--owned", action="store_true",
1043                      help="--freeze-real) assert owner-recorded Fork-A source
(required)")
1044     ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045                      help="--freeze-real) assert the source contains NO minors
(required)")
1046     ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047     ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
1048     ap.add_argument("--label", default="cleared_real",
1049                      help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050                      "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051     ap.add_argument("--source-note", dest="source_note", default="",
1052                      help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053                      "attribution back-channel, so it never reaches the committed
provenance "
1054                      "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055     ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056                      help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057     ap.set_defaults(time_origin_reset=True)
1058     ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059                      help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060     args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062     fixtures_dir: Path = args.fixtures_dir
1063
1064     if args.freeze_synthetic:
1065         slugs = freeze_synthetic(fixtures_dir)
1066         print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
1067         print(f" -> {fixtures_dir}")
1068         return 0
1069
1070     if args.refresh_snapshot:
1071         if not args.slug and not args.all:
1072             ap.error("--refresh-snapshot requires --slug S or --all")
1073             # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
1074             # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
1075             if not (args.reason and args.reason.strip()):
1076                 ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
1077                         "regression; record the deliberate reason and review the visible
diff ($6).")
1078             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
1079             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}"
1080                   f" (reason: {args.reason})")
1081             return 0
1082
1083     if args.freeze_real:
1084         missing = [n for n, v in (("--trajectory", args.trajectory),
1085                                  ("--labels", args.labels),
1086                                  ("--fps", args.fps),
1087                                  ("--frame-width", args.frame_width)) if v is None]
1088         if missing:

```

```

1089         ap.error("--freeze-real requires " + ", ".join(missing))
1090     try:
1091         expected = freeze_real_fixture(
1092             args.label, args.trajectory, args.labels,
1093             license=args.license_ if args.license_ is not None else "",
1094             minors_free=args.minors_free, owned=args.owned,
1095             fps=args.fps, frame_width=args.frame_width,
1096             fixtures_dir=fixtures_dir, slug=args.slug,
1097             time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1098     except RefusalError as e:
1099         # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
1100         zero.
1101         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
1102             file=sys.stderr)
1103         return 2
1104     print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'"
1105         "
1106         f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
1107     print(" §6: behaviour-locking, NOT correctness; non-attributable only relative
1108         to the "
1109         "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
1110         (P2).")
1111     return 0
1112 # --check
1113 return _cmd_check(fixtures_dir)
1114
1115 if __name__ == "__main__":
1116     raise SystemExit(main())

```

42. assistant (2026-06-18T01:34:46.152Z)

This is comprehensive. Let me now scrutinize a couple of remaining suspicious spots and verify behavior empirically. Let me re-examine the `bootstrap\_ci` CI index math and the `paired\_delta\_ci` for the case where `ci\_excludes\_zero` could be miscomputed, plus the `\_percentile` and `\_spread` rounding. I'll run a quick test of the statistical functions since they're pure stdlib.

43. user (2026-06-18T01:34:51.319Z)

```

all-equal: (0.5, 0.5)
mixed: (0.0, 1.0)
n=6: (0.21666666666666667, 0.48333333333333334)

```

44. assistant (2026-06-18T01:34:57.213Z)

Bootstrap looks fine. Now let me scrutinize the one area I flagged as most promising: the R4 `\_trim\_inactive\_tail` and the interaction between `merge\_gap` frame computation and window boundaries. Let me also carefully re-examine `\_points\_in` in rally_gate which uses `start \* fps <= p.frame <= end \* fps`. There's a subtle issue: `gate\_windows` is called from `windows\_for` in rally_seg_eval with `ivs` derived from `windows\_to\_intervals`. Those window start/end are `frame/fps + chunk\_start`. With chunk_start=0, the boundaries are exact. But float rounding `start\*fps` could exclude the boundary frame. Let me test concretely.

45. user (2026-06-18T01:34:59.962Z)

```

frame=100 s=1.6683350016683351 s*fps=100.0 includes_frame=True
frame=250 s=4.170837504170838 s*fps=250.00000000000003 includes_frame=False
frame=1799 s=30.01334668001335 s*fps=1799.0 includes_frame=True
frame=3000 s=50.050050050050054 s*fps=3000.0 includes_frame=True

```

46. assistant (2026-06-18T01:35:09.008Z)

Interesting ? for `frame=250`, `s\*fps = 250.00000000000003 > 250`, so the boundary frame would be EXCLUDED by `start\*fps <= p.frame`. This is a floating-point edge effect in `\_points\_in`. But is it a real defect? This is a window-boundary inclusivity issue: a single boundary frame may be excluded from the net-crossing count. Given it affects at most the first/last frame of a window and the gate counts many crossings, the practical impact is negligible ? this is borderline noise, not a maintainer-recognized defect. I'll keep it in mind but it's low-confidence.

Let me look at something more impactful. Let me re-examine the `kfold\_indices` round-robin in calibration.py. `folds = [list(range(i, n, k)) for i in range(k)]`. This is a round-robin assignment. With `k = max(2, min(k, n))`. The fold sizes are balanced. Each test fold and train. This looks correct.

Let me now look closely at a potential real issue in `over\_seg\_guard`'s `\_aligned` ? and the `paired\_delta\_ci` when `a\_values`/`b\_values` have different lengths (it uses `min`, dropping the tail silently). Let me also examine the `mean\_average\_precision` and the AP precision-envelope loop. Let me check the `\_percentile` function edge ? when `p=100`, `k = (len-1)\*100/100 = len-1`, `lo=len-1`, `hi=min(len, len-1)=len-1`. Fine.

Let me look at the remaining unread eval files: batch.py, per_user_regression.py, calibrate_local.py, distill_local.py (build_candidates used by fixtures), and rally_seq_proto.py.

47. user (2026-06-18T01:35:09.288Z)

```
1      """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3      Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4      further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5      using a few resolution/fps-INVARIANT local features, so the model transfers across
6      videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8      Flow (all local at runtime):
9          WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10             filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11             active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12             -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37      from backend.eval import windowing as _windowing
38
39      Interval = Tuple[float, float]
40
```

```

41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45     # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48     PROPOSAL = _windowing.PROPOSAL.as_tuple()
49     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
50
51
52     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
53                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
56         vt, mg, mwd = proposal
57         wins = TrackNetRunner.trajectory_to_action_windows(
58             points, fps=fps, frame_width=frame_width,
59             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60         intervals = [(w["start"], w["end"]) for w in wins]
61         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62         X: List[List[float]] = []
63         for w, g in zip(wins, gated):
64             dur = max(1e-6, w["end"] - w["start"])
65             win_frames = max(1.0, dur * fps)
66             X.append([
67                 dur, # rally length (sec) ?
fps-invariant
68                 float(w["peak_velocity"]), # already normalized by
frame_width
69                 float(w["mean_velocity"]),
70                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
fps-invariant
71                 float(g.crossings), # net crossings (count) ?
the rally signal
72             ])
73         return intervals, X
74
75
76     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
78         vt, mg, mwd = BASELINE_LOCAL
79         wins = TrackNetRunner.trajectory_to_action_windows(
80             points, fps=fps, frame_width=frame_width,
81             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82         return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
91                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}

```



```

92
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95                             threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104             model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107               f"({len(videos)} videos, IoU={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]
119                 X = [row for v in others for row in v["X"]]
120                 y = [t for v in others for t in v["y"]]
121                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123                 sc = score(preds, held["gts"], iou)
124                 bsc = score(held["baseline"], held["gts"], iou)
125                 clf.append(sc)
126                 base.append(bsc)
127                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128                       f"    |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129             def mean(xs, a):
130                 return sum(getattr(s, a) for s in xs) / len(xs)
131             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
132                   f"F1={mean(clf, 'f1'):.3f} |  baseline P={mean(base, 'precision'):.3f} "
133                   f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137             # Final model on ALL videos (for production / running on a fresh held-out video).
138             Xall = [row for v in videos for row in v["X"]]
139             yall = [t for v in videos for t in v["y"]]
140             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141             assert final.w is not None # set by fit()
142             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144             if model_out:
145                 final.save(model_out)
146                 print(f"final model (all videos) -> {model_out}")
147                 result["model_out"] = model_out
148             return result
149
150
151     def main() -> None:

```

```

152     ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153     ap.add_argument("--manifest", required=True,
154                     help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155     ap.add_argument("--iou", type=float, default=0.5)
156     ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157     ap.add_argument("--model-out", default="output/local_rally_model.json")
158     args = ap.parse_args()
159     with open(args.manifest, encoding="utf-8") as f:
160         specs = json.load(f)
161     run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

48. assistant (2026-06-18T01:35:16.114Z)

Now let me carefully scrutinize the ``paired_sign_test`` two-sided p-value computation in `eval_stats.py` ? this is a common source of statistical bugs. The exact two-sided binomial test at $p=0.5$ should be ``2 * P(X <= k)`` where ``k = min(wins, losses)``, but capped at 1.0. Let me verify the math is correct for a critical edge case: when ``wins == losses``, ``k = wins``, and ``tail = P(X <= wins)``. Since ``wins = losses = n_eff/2``, ``P(X <= n_eff/2)`` already exceeds 0.5, so ``2*tail > 1``, capped at 1.0. Correct (no evidence either way ? $p=1.0$).

But there's a subtle issue. The exact two-sided sign test p-value formula ``2 * P(X <= k)`` is the standard formula, but it double-counts the central probability when ``wins==losses``. Capping at 1.0 handles that. This is the standard implementation. Fine.

Let me now look at the ``per_user_regression.py``, ``batch.py``, and ``calibrate_local.py`` for completeness, plus the ``served_gate_a.py``.

49. user (2026-06-18T01:35:17.531Z)

```

1     """Per-user NO-REGRESSION guard ? the missing per-user analogue of `regression.py`.
2
3     The shared golden set has a disciplined no-regression baseline (`regression.py` +
4     ``regression_baseline.json``): a candidate is gated so a global change can never
silently
5     make the *shared* corpus worse. Personalization has no such rail ? a deliberately-tuned
6     per-user model "has no shared baseline to regress against", so a per-user retrain or a
7     baseline upgrade could quietly make ONE user worse than the model they're already
running.
8     This module builds that missing per-user regression baseline, mirroring the shared
discipline.
9
10    The decision it makes is deliberately narrow and CONSERVATIVE ? it is a *guarantee
against
11    regression*, not a promotion decision (that is `per_user_gate.should_promote_for_user`):
12
13    * **CONTROL** = the user's *frozen incumbent* ? the variant/config currently active
FOR THAT
14    USER. For a brand-new user with nothing personalized yet, the incumbent simply IS
the
15    shipped baseline. This is the thing we must never regress below.
16    * **CANDIDATE** = the proposed replacement (a per-user retrain, or a global baseline
upgrade
17    rolled out to this user).
18
19    Both are evaluated on the user's own held-out videos (`experiment.evaluate_variant`,
LOVO-honest
20    for a learned recipe ? no new scoring math), paired by video, and summarised with the

```

```

shipped
21     honest statistics (`eval_stats.paired_delta_ci` ? bootstrap ?F1 CI + exact sign test).
Then:
22
23     * regression ? the candidate is significantly worse: the paired ?F1 (candidate ?
control)
24     95% CI excludes 0 on the WRONG side (`ci_high < 0`). ? REJECT, keep the incumbent.
25     * clear improvement ? the candidate is significantly better: the CI excludes 0
on the
26     right side AND the sign test agrees (candidate wins on strictly more videos than it
loses).
27     ? ACCEPT the swap.
28     * tie / insufficient evidence ? the CI spans 0 (the small-N near-null regime):
there is no
29     evidence the swap helps, so we never take the risk. ? KEEP THE INCUMBENT.
30
31     So we only ACCEPT a swap we can defend and only ever toward a measured improvement;
everything
32     ambiguous keeps the incumbent. That is strictly stronger than "don't regress" (it also
refuses
33     risk-neutral churn), which is exactly the safety the personalization feature was
missing.
34
35     A small per-user baseline record is keyed by a `user_id::stage` string (`user_id`
nullable ?
36     `None` = the local single-user deployment; design-for-north-star, implement-for-
current) and
37     saved/loaded as JSON, a sibling to `regression.py`'s `regression_baseline.json`
approach.
38
39     Pure-CPU, deterministic given `seed` (the only randomness is the CI bootstrap); no GPU
/ torch /
40     cv2 / file I/O beyond the explicit baseline save/load. See `docs/EXPERIMENT_HARNESS.md`
+
41     `docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
42     ""
43     from __future__ import annotations
44
45     import json
46     import logging
47     import os
48     from dataclasses import asdict, dataclass, field
49     from typing import Any, Dict, List, Optional, Sequence
50
51     from backend.eval import eval_stats as _stats
52     from backend.eval import experiment
53     from backend.eval.experiment import Variant, VideoCandidates
54
55     logger = logging.getLogger(__name__)
56
57     __all__ = [
58         "DEFAULT_PER_USER_BASELINE_PATH",
59         "PerUserRegressionResult",
60         "PerUserBaseline",
61         "baseline_key",
62         "load_baselines",
63         "save_baseline",
64         "evaluate_no_regression",
65     ]
66
67     # Tracked location (NOT under the gitignored output/), sibling to
68     # regression.DEFAULT_BASELINE_PATH so a fresh checkout / CI can load a committed per-
user
69     # baseline. NULL user_id (local single-user) keys as the literal "local".
70     DEFAULT_PER_USER_BASELINE_PATH = os.path.join("eval_baselines",

```

```

"per_user_baseline.json")
71
72
73     # ----- result
74
75     @dataclass
76     class PerUserRegressionResult:
77         """The per-user no-regression decision plus the evidence behind it.
78
79         - ``accept`` ? swap the candidate in for this user (only ever ``True`` on a
*defended*
80         improvement: the ?F1 CI clears 0 on the right side AND the sign test agrees).
81         - ``keep_incumbent`` ? the user keeps their frozen incumbent (control). Always the
exact
82         complement of ``accept`` ? a swap is taken or it is not.
83         - ``regressed`` ? the candidate is *significantly worse* than the incumbent (CI
excludes 0
84         on the wrong side). A True here always implies ``accept=False``.
85         - ``reason`` ? short human-readable explanation, for logging / audit.
86         - the remaining fields surface the paired evidence (per-variant mean held-out F1,
mean ?F1,
87         CI bounds, and the sign test) so a caller can audit exactly why the swap was/
wasn't taken.
88         """
89
90         accept: bool
91         keep_incumbent: bool
92         reason: str
93         n: int
94         control_f1: float
95         candidate_f1: float
96         mean_delta: float
97         ci_low: float
98         ci_high: float
99         ci_excludes_zero: bool
100        regressed: bool
101        sign_test: Dict[str, float] = field(default_factory=dict)
102
103        def as_dict(self) -> Dict[str, Any]:
104            """Plain-dict view (for JSON logging / telemetry)."""
105            return asdict(self)
106
107
108     # ----- per-user baseline
store
109
110     @dataclass
111     class PerUserBaseline:
112         """A snapshotted per-user incumbent record ? the per-user analogue of a regression
baseline.
113
114         Keyed by ``user_id::stage``. ``user_id`` is nullable (``None`` ? the local single-
user
115         deployment, keyed as the literal ``"local"``). ``control_spec_hash`` pins WHICH
incumbent
116         variant the recorded numbers were measured for, so a later check against a different
117         incumbent is detectable rather than silently trusted (mirrors regression.py's
118         iou/detector/gt fingerprint guards).
119         """
120
121         user_id: Optional[str]
122         stage: str
123         control_f1: float
124         control_spec_hash: str
125         n: int

```

```

126         iou: float
127
128     def as_dict(self) -> Dict[str, Any]:
129         return asdict(self)
130
131
132     def baseline_key(user_id: Optional[str], stage: str) -> str:
133         """Stable ``user_id::stage`` key. A NULL ``user_id`` (local single-user) keys as
134         ``local``."""
135         return f"{user_id if user_id is not None else 'local'}::{stage}"
136
137     def load_baselines(path: str = DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
138         """Load the per-user baseline file (returns ``{}`` if absent)."""
139         if not os.path.isfile(path):
140             return {}
141         with open(path, encoding="utf-8") as f:
142             return json.load(f)
143
144
145     def save_baseline(baseline: PerUserBaseline,
146                      path: str = DEFAULT_PER_USER_BASELINE_PATH) -> str:
147         """Snapshot a user's incumbent numbers as the new per-user baseline, keyed
148         ``user_id::stage``.
149
150         Round-trips through ``load_baselines``; returns the key written. Mirrors
151         ``regression.save_baseline`` (atomic-ish read-modify-write of the one JSON file).
152         """
153         baselines = load_baselines(path)
154         key = baseline_key(baseline.user_id, baseline.stage)
155         baselines[key] = baseline.as_dict()
156         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
157         with open(path, "w", encoding="utf-8") as f:
158             json.dump(baselines, f, indent=2, sort_keys=True)
159         logger.info("Saved per-user baseline %s (control_f1=%.3f n=%d)",
160                     key, baseline.control_f1, baseline.n)
161         return key
162
163     # ----- the guard
164
165     def _mean(xs: Sequence[float]) -> float:
166         return sum(xs) / len(xs) if xs else 0.0
167
168
169     def evaluate_no_regression(
170         videos: Sequence[VideoCandidates],
171         control: Variant,
172         candidate: Variant,
173         *,
174         user_id: Optional[str] = None,
175         stage: str = "final",
176         iou: float = 0.5,
177         honest: bool = True,
178         p_threshold: float = 0.05,
179         confidence: float = 0.95,
180         n_boot: int = 2000,
181         seed: int = 0,
182         eps: float = 0.0,
183         baseline_path: Optional[str] = None,
184     ) -> PerUserRegressionResult:
185         """Guard a per-user swap: ACCEPT only if ``candidate`` does NOT regress vs the
186         ``control``.
187
188         ``control`` is the user's FROZEN incumbent (their currently-active variant ? for a

```

```

188 brand-new user this is just the shipped baseline). ``candidate`` is the proposed
189 replacement (a per-user retrain or a baseline upgrade). Both are scored on the
user's own
190 ``videos`` via `experiment.evaluate_variant` (LOVO-honest for a learned recipe ? no
new
191 scoring math), the per-video held-out F1s are paired by video, and the paired ?F1
192 (candidate ? control) is summarised with `eval_stats.paired_delta_ci` (bootstrap CI
+
193 exact sign test). The decision (faithful to the module docstring):
194
195     * **regression** (`ci_high < 0` ? candidate significantly worse): REJECT, keep
incumbent.
196     * **clear improvement** (CI excludes 0 on the right side AND the sign test agrees
?
197     candidate wins on strictly more videos than it loses): ACCEPT the swap.
198     * **tie / insufficient evidence** (CI spans 0): KEEP THE INCUMBENT (conservative;
never
199     accept a risky swap).
200
201 ``user_id``/``stage`` key the baseline record (`user_id=None` ? local single-
user). When
202 ``baseline_path`` is given the incumbent's measured numbers are snapshotted there
(the swap
203 is then evaluated as usual). Deterministic given ``seed``. Raises `ExperimentError`
(from
204 `evaluate_variant`) on unlabeled videos or a learned recipe with too few videos for
LOVO.
205
206 """
207 eval_control = experiment.evaluate_variant(videos, control, iou, honest)
208 eval_candidate = experiment.evaluate_variant(videos, candidate, iou, honest)
209
210 # Per-video held-out F1, video-aligned (evaluate_variant iterates ``videos`` in
order, so
211 # both lists are pairable by position ? exactly how honest_summary pairs B ? A).
212 control_f1s: List[float] = [p["f1"] for p in eval_control["per_video"]]
213 candidate_f1s: List[float] = [p["f1"] for p in eval_candidate["per_video"]]
214 n = min(len(control_f1s), len(candidate_f1s))
215
216 control_mean = _mean(control_f1s)
217 candidate_mean = _mean(candidate_f1s)
218
219 if n == 0:
220     # No paired videos at all ? no evidence either way ? keep the incumbent.
221     sign_test: Dict[str, float] = _stats.paired_sign_test([], eps)
222     result = PerUserRegressionResult(
223         accept=False, keep_incumbent=True,
224         reason="no held-out videos to evaluate; keeping incumbent",
225         n=0, control_f1=control_mean, candidate_f1=candidate_mean,
226         mean_delta=0.0, ci_low=0.0, ci_high=0.0, ci_excludes_zero=False,
227         regressed=False, sign_test=sign_test,
228     )
229 else:
230     delta = _stats.paired_delta_ci(
231         control_f1s, candidate_f1s,
232         confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
233     )
234     mean_delta = float(delta["mean_delta"])
235     ci_low = float(delta["ci_low"])
236     ci_high = float(delta["ci_high"])
237     ci_excludes_zero = bool(delta["ci_excludes_zero"])
238     sign_test = delta["sign_test"]
239     sign_p = float(sign_test.get("p_value", 1.0))
240     sign_wins = float(sign_test.get("wins", 0.0))
241     sign_losses = float(sign_test.get("losses", 0.0))

```

```

242         # A regression = the candidate is significantly WORSE: the ?F1 CI excludes 0
entirely
243         # on the wrong side (its whole high end is below 0). Never accept such a swap.
244         regressed = ci_excludes_zero and ci_high < 0.0
245         # A defended improvement = the CI clears 0 on the RIGHT side AND the sign test
agrees
246         # (more candidate-wins than losses, significant) ? the same promotion-grade bar
the
247         # global eval uses, applied per user. Everything else is a tie / insufficient
evidence.
248         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
249         improved = ci_excludes_zero and ci_low > 0.0 and sign_agrees
250
251         if regressed:
252             accept = False
253             reason = (
254                 f"REGRESSION: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] "
255                 f"is entirely below 0 (n={n}); keeping incumbent"
256             )
257         elif improved:
258             accept = True
259             reason = (
260                 f"accepted: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] excludes
0 "
261                 f"(improvement), sign test {int(sign_wins)}-{int(sign_losses)} "
262                 f"(p={sign_p:.4f} <= {p_threshold}); swapping in candidate"
263             )
264         else:
265             accept = False
266             reason = (
267                 f"tie / insufficient evidence: candidate ?F1 95% CI "
268                 f"[{ci_low:.4f}, {ci_high:.4f}] spans 0 (n={n}); keeping incumbent "
269                 f"(no defended improvement, never a risky swap)"
270             )
271
272         result = PerUserRegressionResult(
273             accept=accept, keep_incumbent=not accept, reason=reason,
274             n=n, control_f1=control_mean, candidate_f1=candidate_mean,
275             mean_delta=mean_delta, ci_low=ci_low, ci_high=ci_high,
276             ci_excludes_zero=ci_excludes_zero, regressed=regressed, sign_test=sign_test,
277         )
278
279         # Snapshot the incumbent's measured numbers as this user's baseline when asked ? the
280         # per-user analogue of regression.save_baseline. Does not change the decision above.
281         if baseline_path is not None:
282             save_baseline(
283                 PerUserBaseline(
284                     user_id=user_id, stage=stage,
285                     control_f1=control_mean, control_spec_hash=control.spec_hash(),
286                     n=n, iou=iou,
287                 ),
288                 path=baseline_path,
289             )
290
291         return result
292

```

50. user (2026-06-18T01:35:17.800Z)

```

1         """Batch evaluation across a collector export manifest.
2
3         Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4         sports-data-collector's `curate export`, (optionally) process each video through
5         a segmenter so detections land in the indexer DB, then score every video's
6         detected rallies against its ground-truth CSV and aggregate the results.

```

```

7
8 This module holds the *pure* pieces (path resolution, stage selection, metric
9 aggregation) so they can be unit-tested without touching video files, the GPU,
10 or any API. The orchestration/side-effects live in `scripts/run_phase3_eval.py`.
11 """
12
13 import os
14 from typing import Dict, List, Optional
15
16
17 # Segmenters that only ever populate the `final` play_segments table (no raw
18 # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19 _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22 def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23     """Resolve a manifest `local_path` to an absolute path.
24
25     The collector stores paths like ``./data/videos\\shuttle1.mp4`` relative
26     to its own repo root and with Windows separators. Normalise separators, strip
27     a leading ``./``, and resolve relative paths against `videos_root` (the
28     collector root). Returns None for a missing/empty path.
29     """
30     if not local_path:
31         return None
32     p = local_path.replace("\\", "/")
33     if p.startswith("./"):
34         p = p[2:]
35     # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36     # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37     # This keeps collector manifest paths working in tests and mixed environments.
38     if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39         return os.path.normpath(p)
40     return os.path.normpath(os.path.join(videos_root, p))
41
42
43 def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44     """Pick which prediction stages to score.
45
46     ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47     but only ``final`` for segmenters that don't emit candidates. An explicit
48     request ('candidates' | 'final' | 'both') overrides the heuristic.
49     """
50     if requested == "both":
51         return ["candidates", "final"]
52     if requested in ("candidates", "final"):
53         return [requested]
54     # auto
55     if segmenter in _FINAL_ONLY_SEGMENTERS:
56         return ["final"]
57     return ["candidates", "final"]
58
59
60 def _safe_div(num: float, den: float) -> float:
61     return num / den if den else 0.0
62
63
64 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65     """Aggregate per-video stage scores into corpus-level metrics.
66
67     `per_video` is a list of dicts shaped like `report_to_dict` output (each has
68     `stages[stage]["score"]` with tp/fp/fn/mean_iou/...). For each stage we
69     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70     a TP-weighted mean IoU, plus the macro mean of per-video F1.
71     """

```



```

72 out: Dict[str, dict] = {}
73 for stage in stages:
74     tp = fp = fn = 0
75     iou_weighted = 0.0
76     f1s: List[float] = []
77     n_videos = 0
78     for rep in per_video:
79         sr = rep.get("stages", {}).get(stage)
80         if not sr:
81             continue
82         s = sr["score"]
83         tp += s["true_positives"]
84         fp += s["false_positives"]
85         fn += s["false_negatives"]
86         iou_weighted += s["mean_iou"] * s["true_positives"]
87         f1s.append(s["f1"])
88         n_videos += 1
89     precision = _safe_div(tp, tp + fp)
90     recall = _safe_div(tp, tp + fn)
91     out[stage] = {
92         "videos": n_videos,
93         "tp": tp, "fp": fp, "fn": fn,
94         "micro_precision": precision,
95         "micro_recall": recall,
96         "micro_f1": _safe_div(2 * precision * recall, precision + recall),
97         "macro_f1": _safe_div(sum(f1s), len(f1s)),
98         "mean_iou": _safe_div(iou_weighted, tp),
99     }
100 return out
101
102
103 def format_aggregate(agg: Dict[str, dict]) -> str:
104     """Human-readable corpus-level summary table."""
105     lines = [
106         "=" * 78,
107         " CORPUS AGGREGATE",
108         "=" * 78,
109         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
110         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
111         "-" * 78,
112     ]
113     for stage, a in agg.items():
114         lines.append(
115             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
116             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
117             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
118         )
119     lines.append("=" * 78)
120     return "\n".join(lines)
121

```

51. assistant (2026-06-18T01:35:22.482Z)

I've now covered the bulk of the eval + pipeline windowing numeric/statistical surface. Let me do one final focused pass on the `\_trim\_inactive\_tail` sliding-window logic, which is the most algorithmically intricate windowing code and a prime off-by-one candidate. Let me trace it with a concrete example to verify correctness.

52. user (2026-06-18T01:35:29.604Z)

```

last frame kept: 38 len 39
all fast last: 39 len 40
all slow last: 39 len 40

```

53. assistant (2026-06-18T01:35:41.032Z)

Wait ? for the mixed group (30 fast frames 0-29, then 30 slow frames 30-59), the trim kept frame 38, len 39. That means it kept 9 slow frames past the rally end. Let me examine this more carefully ? is this correct behavior? The trailing window is `trail_frames=15`. At frame `i=38`, the window covers frames `(38-15, 38] = (23, 38]`, i.e. frames 24..38. Of those, frames 24-29 are fast (6 fast) and 30-38 are slow (9 slow), window size 15. Density = $6/15 = 0.4 \geq 0.34$, so it's "dense". At frame 39: window `(24,39] = 25..39`, fast=25..29 (5), size 15, density $5/15=0.333 < 0.34$? not dense. So `last_dense=38`.

This is the documented behavior (trailing-window density), working as designed ? the window keeps frames until the trailing density drops. Not a bug, just the inherent lag of a trailing-window detector. Correct per spec.

Let me verify there isn't a subtle issue in the sliding window where `frames` may have gaps (non-contiguous frame indices after gap-splitting). The window condition uses actual frame values, which handles gaps. Good.

Now let me double-check one thing in `merge_split_report` that I want to be confident about: the `complex` component case. A `(>1, >1)` tangled component contributes nothing to `gts_merged` or `gts_split`, so those GTs are uncounted in both rates. That's a deliberate design choice (documented), not a bug.

Let me verify the `over_seg_report`/`merge_count` consistency with `over_seg_guard`. In `over_seg_guard`, when `use_rates=False`, the cost uses `merge_count`/`split_count`. The `_cost` reads `row["merge_count"]` and `row["split_count"]`. In `rally_seg_eval.score_intervals`, the row has `merge_count`/`split_count` from `osr` (over_seg_report) AND `merge_rate`/`split_rate` from `ms`. Both present. Consistent.

Let me do a final check on the `average_precision` monotone envelope and integration. I want to confirm the recall steps are handled when multiple predictions share the same recall (no new GT matched ? recall flat, skipped correctly).

54. user (2026-06-18T01:35:44.948Z)

```
perfect AP@0.5: 1.0
FP-first AP@0.5: 0.6666666666666666
mAP: 1.0
```

55. assistant (2026-06-18T01:35:57.159Z)

AP math is correct (perfect=1.0; FP-first gives the expected 0.667 via the precision envelope).

I've now thoroughly cross-cut the eval + pipeline windowing surface by bug class: division-by-zero (all guarded with `if ... else 0.0`), `n=0/1` means (bootstrap/sign-test handle these), NaN/inf (sigmoid clipped, `pct_improvement` guards `before<=0`), `fps/frame?time` rounding, interval boundary inclusivity, IoU/tolerance match edges, bootstrap/sign-test/CI math, and cap/trim boundary conditions.

The one floating-point edge I found in `rally_gate._points_in` (`start * fps <= p.frame` can exclude a boundary frame due to FP error, e.g. frame 250 at fps 59.94) is real but its practical impact is negligible ? it can drop at most the single boundary frame of a window from a net-crossing count that requires multiple crossings, and it never changes a keep/drop decision in any realistic scenario. A maintainer would treat this as noise, not a defect. Given the precision-over-recall mandate and that this doesn't reach a concrete consequential trigger, I will not report it as a finding.

After reading the code carefully across all these files, I found no concrete, trigger-reachable defect that a maintainer would call a real bug. The statistical and windowing code is unusually careful ? every aggregate guards its empty/singleton cases, the percentile/CI index math is bounds-clamped, the recursive split/chop functions provably terminate, and the tolerance/IoU matchers correctly require positive overlap. The empty list is the honest answer here.

56. user (2026-06-18T01:35:58.000Z)

Structured output provided successfully

57. assistant (2026-06-18T01:36:14.415Z)

I completed an adversarial cross-cut of the eval + pipeline windowing surface by the numeric/statistical/off-by-one bug class and found no reportable defect.

What I reviewed

Read in full and traced concretely:

- `backend/eval/tolerance\_metrics.py` ? strict 1:1 tolerance match (greedy + Hungarian), `\_percentile`, over-seg guard rate math, F1 zero-guards
- `backend/eval/metrics.py` ? `interval\_iou` degeneracy/zero-overlap guard, greedy matcher, P/R/F1/mean-error denominators
- `backend/eval/eval\_stats.py` ? `bootstrap\_ci` (n=0/1 + percentile index clamping), `paired\_sign\_test` exact two-sided binomial, `paired\_delta\_ci`, LOGO folds
- `backend/pipeline/detectors/tracknet\_runner.py` ? `trajectory\_to\_action\_windows`, the R4 `\_trim\_inactive\_tail` sliding window, `\_split\_overlong\_groups` (hard-cap midpoint), `\_blob\_fallback\_chop` ? verified termination and boundary inclusivity
- `backend/pipeline/detectors/rally\_gate.py` ? `\_spread`/`\_median` percentile indices, `count\_net\_crossings`, `\_points\_in` frame/time round-trip
- `backend/eval/` ? segmentation_metrics (AP envelope + mAP), calibration (kfold/LOVO), classifier (logreg std guard), score_calibration (Platt/isotonic/PAVA/ECE binning), splits, gt_loader, annotations (timestamp parsing), promotion, per_user_gate, per_user_eval, per_user_regression, experiment, harness, regression, golden_fixtures, batch, distill_local, calibrate_wasb, gemini_refine, eval_partial_labels

Excluded per instructions: ablation.py, fusion_compare.py, fusion_audio.py, scripts/nightly_regression.py.

Findings: none

Every aggregate guards its empty/singleton case (`... if den else 0.0`), the bootstrap/percentile index math is bounds-clamped, the sign-test caps the two-sided tail at 1.0, the recursive split/chop functions provably make progress (`split\_i ? range(1, len(g))` keeps both halves non-empty and strictly smaller), and the IoU/tolerance matchers correctly require strictly-positive overlap. I empirically confirmed bootstrap CIs, AP envelope, and the R4 trailing-window trim behave as documented.

One borderline observation I deliberately did **not** report: in `rally\_gate.\_points\_in`, the boundary test `start \* fps <= p.frame` can exclude a single boundary frame due to float error (e.g. frame 250 at fps 59.94 gives `start\*fps = 250.00000000000003`). It is real but inconsequential ? it can drop at most one boundary frame from a multi-crossing count and never flips a keep/drop decision ? so a maintainer would treat it as noise, not a defect. Per the precision-over-recall mandate, the empty list is the honest answer.

The read-only worktree (`origin/master`, HEAD `4bd9d2d`) was not modified.